

References — Containerization

Contents

1 External sources

- 1.1 S1
- 1.2 S2
- 1.3 S3
- 1.4 S4
- 1.5 S5
- 1.6 S6
- 1.7 S7
- 1.8 S8
- 1.9 S9
- 1.10 S10
- 1.11 S11
- 1.12 S12
- 1.13 S13
- 1.14 S14
- 1.15 S15
- 1.16 S16
- 1.17 S17
- 1.18 S18
- 1.19 S19
- 1.20 S20
- 1.21 S21
- 1.22 S22
- 1.23 S23
- 1.24 S24
- 1.25 S25
- 1.26 S26
- 1.27 S27
- 1.28 S28
- 1.29 S29
- 1.30 S30
- 1.31 S31
- 1.32 S32
- 1.33 S33

1.34 S34

1.35 S35

1.36 S36

1.37 S37

1.38 S38

1.39 S39

1.40 S40

1.41 S41

1.42 S42

1.43 S43

1.44 S44

1.45 S45

1.46 S46

1.47 S47

1.48 S48

1.49 S49

1.50 S50

1.51 S51

1.52 S52

1.53 S53

1.54 S54

1.55 S55

1.56 S56

1.57 S57

1.58 S58

1.59 S59

1.60 S60

1.61 S61

1.62 S62

1.63 S63

1.64 S64

1.65 S65

1.66 S66

1.67 S67

1.68 S68

1.69 S69

1.70 S70

1.71 S71

1.72 S72

1.73 S73

1.74 S74

1.75 S75

1.76 S76

1.77 S77

1.78 S78

1.79 S79

1.80 S80

1.81 S81

1.82 S82

1.83 S83

1.84 S84

1.85 S85

1.86 S86

2 Vault sources

2.1 V1 – Container Isolation and Distribution via Docker and Podman

2.2 V2 – Docker and Podman Containers

2.3 V3 – Containers with Docker and Podman

2.4 V4 – Docker and Podman as implementations of Linux container primitives

2.5 V5 – Linux containers and the Docker/Podman engines built on them

2.6 V6 – Containers as an Assembly of Independent Kernel Mechanisms

2.7 V7 – Containers as an Assembly of Independent Kernel Mechanisms

2.8 V8 – The Kernel Mechanisms and Security Trade-offs of Docker and Podman

2.9 V9 – Linux containers as kernel-requested process isolation, implemented by Docker and Podman

2.10 V10 – Containers implemented via Docker and Podman

2.11 V11 – Linux Containers: Kernel Mechanisms and the Docker/Podman Architecture

2.12 V12 – Container isolation mechanics and the Docker-Podman architecture split

2.13 V13 – Containers implemented with Docker and Podman

2.14 V14 – Container Isolation Under Docker and Podman

2.15 V15 – Linux Containers as Kernel Mechanism, Not a Kernel Object

2.16 V16 – Containers as a bundle of independent kernel mechanisms

2.17 V17 – Containers with Docker and Podman

2.18 V18 – Linux Containers as Implemented by Docker and Podman

2.19 V19 – Copy-on-write (CoW)

2.20 V20 – Copy-on-write as one mechanism across scales

2.21 V21 – The System Call Boundary Between Programs and the Kernel

2.22 V22 – The Leaky Boundary Between OS Abstractions and the Machine

2.23 V23 – Where operating-system abstractions leak

2.24 V24 – Dev Containers

2.25 V26 – Processes, threads, and schedulers on Linux and Windows

2.26 V27 – Linux containers, and the Docker/Podman architectures built on them

2.27 V28 – the Linux kernel mechanisms and history underlying Docker and Podman

2.28 V29 – Containers with Docker and Podman

2.29 V30 – Container Isolation and the Docker/Podman Architecture Split

2.30 V31 – Containers as Linux Kernel Primitives Assembled by Docker and Podman

2.31 V32 – The Five-Mechanism Model of a Linux Container

2.32 V33 – Containers as a Composition of Independent Kernel Primitives

2.33 V34 – Docker and Podman as Compositions of Independent Kernel Mechanisms

2.34 V35 – The Container as a Composition of Independent Kernel Mechanisms

2.35 V36 – Docker and Podman as Configurations of Kernel Primitives

2.36 V37 – Container Isolation and Distribution via Docker and Podman

2.37 V38 – Docker and Podman Container Mechanics

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

v2 · e497e0f7b · built 2026-09-29

This document lists every source the talk cites. [S<n>] entries are external sources a reader can open. [V<n>] entries are notes from the author's private research vault. The vault is not public, so each [V] entry carries a summary of the note and every external source the note draws on.

1 External sources

1.1 S1

- **Type:** Reference
- **Credibility:** High — the canonical kernel interface documentation, maintained alongside the kernel itself.
- **Relevance:** Part III. The kernel's own definition of a namespace, the eight-type table, and the `clone / unshare / setns` contract that makes a namespace an ordinary syscall a student can trace.
- **URL:** <https://man7.org/linux/man-pages/man7/namespaces.7.html>

The spans relied on: "A namespace wraps a global system resource in an abstraction that makes it appear to the processes within the namespace that they have their own isolated instance of the global resource"; the eight types with their `CLONE_NEW*` flags; `clone(2)` "creates a new process... if the *flags* argument specifies one or more of the `CLONE_NEW*` flags... then new namespaces are created"; `unshare(2)` "moves the calling process to a new namespace"; `setns(2)` "allows the calling process to join an existing namespace... specified via a file descriptor"; and, on the `/proc/pid/ns/` entries, that each is a symbolic link whose content is "a string containing the namespace type and inode number", that "opening one of the files in this directory (or a file that is bind mounted to one of these files) returns a file handle for the corresponding namespace"; and that bind mounting one of these files "keeps the corresponding namespace of the process specified by *pid* alive even if all processes currently in the namespace terminate".

1.2 S2

- **Type:** Reference
- **Credibility:** High — canonical kernel interface documentation for the control-group interface.
- **Relevance:** Part III. What a control group is and what it does, plus the v1-to-v2 unified-hierarchy change that rootless containers depend on.
- **URL:** <https://man7.org/linux/man-pages/man7/cgroups.7.html>

The spans relied on: control groups are "a Linux kernel feature which allow processes to be organized into hierarchical groups whose usage of various types of resources can then be limited and monitored"; under v2 "all mounted controllers reside in a single unified hierarchy"; the no-internal-processes rule, "processes may reside only in leaf nodes (cgroups that do not themselves contain child cgroups)"; and that cgroups version 1 appeared in "Linux 2.6.24".

1.3 S3

- **Type:** Reference
- **Credibility:** High — canonical kernel interface documentation for the privilege model.
- **Relevance:** Parts II, III and VI. The definition of a capability, the scope of `CAP_SYS_ADMIN`, `CAP_NET_RAW` and `CAP_SYS_PTRACE` — the last carries the whole tracing result in Part III.
- **URL:** <https://man7.org/linux/man-pages/man7/capabilities.7.html>

The spans relied on: "Starting with Linux 2.2, Linux divides the privileges traditionally associated with superuser into distinct units, known as *capabilities*, which can be independently enabled and disabled"; and `CAP_SYS_PTRACE` "permits tracing arbitrary processes, inspecting sensitive process information through `/proc` ... and transferring data to/from other processes' memory". `CAP_NET_RAW` is listed as permitting a process to "Use RAW and PACKET sockets", which is the span Part III relies on for the Docker-versus-Podman default difference. The page also warns that `CAP_SYS_ADMIN` grants "overloaded functionality" and advises kernel developers not to add new features to it, and it documents the `CapEff` and related lines of `/proc/[pid]/status` as where a process's capability sets can be read.

1.4 S4

- **Type:** Reference
- **Credibility:** High — canonical kernel interface documentation for the syscall filter.
- **Relevance:** Part III. Strict mode against filter mode, and the rule that installing a filter needs `CAP_SYS_ADMIN` or the `no_new_privs` bit.
- **URL:** <https://man7.org/linux/man-pages/man2/seccomp.2.html>

The spans relied on: `SECCOMP_SET_MODE_STRICT` permits only `read(2)`, `write(2)`, `_exit(3)` and `sigreturn(2)`; and installing a filter requires "either the calling thread must have the **CAP_SYS_ADMIN** capability in its user namespace, or the thread must already have the `no_new_privs` bit set".

1.5 S5

- **Type:** Reference
- **Credibility:** High — vendor documentation for the vendor's own default seccomp profile, and it states the kernel-version qualifier against its own received wisdom.
- **Relevance:** Parts II and III. The size of the default allowlist, and the qualifier that corrects the folklore about `ptrace` being blocked.
- **URL:** <https://docs.docker.com/engine/security/seccomp/>

The spans relied on: the default profile "disables around 44 system calls out of 300+"; `mount` is denied with the reason "Deny mounting, already gated by `CAP_SYS_ADMIN`"; `ptrace` carries the qualifier "Blocked in Linux kernel versions before 4.8 to avoid seccomp bypass"; and the note that "Tracing/profiling *arbitrary* processes is already blocked by dropping `CAP_SYS_PTRACE`".

1.6 S6

- **Type:** Reference
- **Credibility:** High — vendor documentation for the default storage driver, and the source of the layer vocabulary every engine uses.
- **Relevance:** Part III. The `lowerdir / upperdir / merged` model, the whole-file `copy_up` cost, the 128-layer limit, and page-cache sharing across containers.
- **URL:** <https://docs.docker.com/engine/storage/drivers/overlayfs-driver/>

The spans relied on: "OverlayFS layers two directories on a single Linux host and presents them as a single directory... OverlayFS refers to the lower directory as `lower-dir` and the upper directory as `upperdir`. The unified view is exposed through its own directory called `merged`"; "The first time a container writes to an existing file... the `overlay2` driver performs a `copy_up` operation to copy the file from the image (`lower-dir`) to the container (`upperdir`)"; "OverlayFS works at the file level rather than the block level... all OverlayFS `copy_up` operations copy the entire file, even if the file is large and only a small part of it's being modified"; "Multiple containers accessing the same file share a single page cache entry for that file"; and that the `overlay2` driver "supports up to 128 lower OverlayFS layers".

1.7 S7

- **Type:** Primary source
- **Credibility:** High — the normative standard itself, published by the Open Container Initiative.
- **Relevance:** Part III. The Linux section of `config.json` is a direct encoding of the five mechanisms, which is the deck's central claim in machine-readable form.
- **URL:** <https://github.com/opencontainers/runtime-spec/blob/main/config-linux.md>

The spans relied on: the `namespaces` array whose `type` field takes `pid`, `network`, `mount`, `ipc`, `uts`, `user`, `cgroup` or `time`, with an optional `path` to join an existing namespace; `cgroupsPath` and the `resources` object; the `seccomp` object with `defaultAction`, `architectures` and a `syscalls` array; and `maskedPaths`, which "will mask over the provided paths inside the container so that they cannot be read".

1.8 S8

- **Type:** Primary source
- **Credibility:** High — the normative standard itself.
- **Relevance:** Parts I, II and IV. What an image is, stated by the specification: a manifest plus content-addressed layer archives, which is the distribution half of the deck's argument.
- **URL:** <https://github.com/opencontainers/image-spec/blob/main/spec.md>

The span relied on: the *image manifest* is "a document describing the components that make up a container image", containing "the content-addressable identity of one or more filesystem layer changeset archives that will be unpacked to make up the final runnable filesystem".

1.9 S9

- **Type:** Primary source
- **Credibility:** High — the standards body's own account of its founding date and founding members.
- **Relevance:** Part VII. The date the image format and the runtime contract stopped being one vendor's implementation detail, which is why a second engine can be a drop-in.
- **URL:** <https://opencontainers.org/about/overview/>

The span relied on: the OCI was "launched on June 22nd 2015 by Docker, CoreOS and other leaders in the container industry".

1.10 S10

- **Type:** Reference
- **Credibility:** High — vendor documentation describing the vendor's own architecture.
- **Relevance:** Part IV. The client-server shape, the REST API over a UNIX socket, and what `dockerd` owns.
- **URL:** <https://docs.docker.com/get-started/docker-overview/>

The spans relied on: "Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers"; the two communicate "through a REST API, over UNIX sockets or a network interface"; and `dockerd` "listens for Docker API requests and manages Docker objects such as images, containers, networks, and volumes".

1.11 S11

- **Type:** Reference
- **Credibility:** High — vendor documentation for the vendor's own rootless mode, including its prerequisites.
- **Relevance:** Part IV. Docker's rootless mode keeps the daemon and moves it into a user namespace, which shows where the architecture is load-bearing.
- **URL:** <https://docs.docker.com/engine/security/rootless/>

The spans relied on: "Rootless mode lets you run the Docker daemon and containers as a non-root user to mitigate potential vulnerabilities in the daemon and the container runtime", by executing "the Docker daemon and containers inside a user namespace"; "with userns-remap mode, the daemon itself is running with root privileges, whereas in rootless mode, both the daemon and the container are running without root privileges"; and the requirement for "at least 65,536 subordinate UIDs/GIDs for the user". The page documents Docker only and carries no claim about Podman, so no Podman identifier-range claim rests on it.

1.12 S12

- **Type:** Reference
- **Credibility:** High — vendor documentation for the feature, and it states its own default and its own support limit.
- **Relevance:** Part IV. The honest, smaller form of the daemonless argument: Docker can keep containers alive across a daemon restart, but not by default.
- **URL:** <https://docs.docker.com/engine/daemon/live-restore/>

The spans relied on: "You can configure the daemon so that containers remain running if the daemon becomes unavailable"; and the restriction that live restore is "only supported when installing patch releases".

1.13 S13

- **Type:** Reference
- **Credibility:** High — the project's own normative manual page.
- **Relevance:** Part IV. Primary-source confirmation of the daemonless design, regular-user operation, and `crun` as the default runtime.
- **URL:** <https://docs.podman.io/en/latest/markdown/podman.1.html>

The spans relied on: Podman is "a fully featured container engine that is a simple daemonless tool" where "most Podman commands can be run as a regular user, without requiring additional privileges"; and `--runtime` takes the "name of the OCI runtime as specified in `containers.conf` or absolute path to the OCI compatible binary used to run containers", where "the default runtime is `crun`".

1.14 S14

- **Type:** Primary source

- **Credibility:** High — the project's own description of its design, at its canonical repository location.
- **Relevance:** Part IV. The privilege ceiling a rootless container cannot exceed.
- **URL:** <https://github.com/podman-container-tools/podman>

The span relied on: "rootless containers will never have more privileges than the user that launched them".

1.15 S15

- **Type:** Blog / editorial
- **Credibility:** High — written by the Podman and container-runtime maintainers about their own implementation; its 2020 storage claim is treated as dated rather than current.
- **Relevance:** Part IV. The rootless start sequence — pause process, subordinate-ID mapping, `common`, userspace networking — which is what daemonless costs and buys.
- **URL:** <https://www.redhat.com/en/blog/behind-scenes-podman>

The spans relied on: Podman joins the pause process's namespace "because a multithreaded program cannot change its user namespace"; and "In rootless Podman, we cannot create full, separate networking for containers, because this feature is not allowed for non-root users", so `slirp4netns` "creates a tap device that is injected inside the new network namespace... and emulates a TCP/IP stack in user space".

1.16 S16

- **Type:** Primary source
- **Credibility:** High — authoritative on its own implementation, but the benchmark is self-reported on a self-selected workload and is named as such on the slide.
- **Relevance:** Part IV. The C-against-Go rationale and the project's own startup and memory figures.
- **URL:** <https://github.com/containers/crun>

The spans relied on: `crun` is "a fast and low-memory footprint OCI Container Runtime fully written in C"; `runc` "re-execs itself and uses a module written in C for setting up the environment before the container process starts"; the 100-sequential-container figures of 1.69 s against 3.34 s; and the 512 KB against 4 MB memory-limit result.

1.17 S17

- **Type:** Blog / editorial
- **Credibility:** High — traces its claim to the containerd source rather than asserting it, and is corroborated on the kernel-version point by S5.
- **Relevance:** Parts II and III. The capability-to-seccomp coupling, and the change that dates the received explanation for `strace` failing.
- **URL:** <https://jvns.ca/blog/2020/04/29/why-strace-doesnt-work-in-docker/>

The spans relied on: in containerd, "if a process has a capability, then it's also given access (through a seccomp rule) to use the system calls that go with that capability"; adding `--cap-add=SYS_PTRACE` allowlists `ptrace`, `process_vm_readv`, `process_vm_writev` and `kcmp` as a set; and "as of Docker 19.03, Docker does actually allow the `ptrace` system calls for kernel versions newer than 4.8".

1.18 S18

- **Type:** Blog / editorial
- **Credibility:** High — a widely cited mechanism-first explainer whose namespace claims match the kernel man pages; it is an explainer rather than a primary source.
- **Relevance:** Parts II and III. The single-sentence correction of the lightweight-VM reading, and the concrete PID-namespace instance Part III opens with.
- **URL:** <https://jvns.ca/blog/2016/10/10/what-even-is-a-container/>

The spans relied on: "these features let you pretend you have something like a virtual machine, except it's not a virtual machine at all, it's just processes running in the same Linux kernel"; and "in a **pid** namespace you become PID 1 and then your children are other processes. All the other programs are gone".

1.19 S19

- **Type:** Academic paper
- **Credibility:** High — the peer-reviewed paper that introduced FreeBSD jails, written by its authors and presented at SANE 2000.
- **Relevance:** Parts I and VII. It states the partial-compartmentalisation argument in 2000, and it is also the origin of the lightweight-VM reading.
- **URL:** <http://phk.freebsd.dk/pubs/sane2000-jail.pdf>

The spans relied on: "In the case of the chroot(2) call, a process's visibility of the file system name-space is limited to a single subtree. However, the compartmentalisation does not extend to the process or networking spaces and therefore both observation of and interference with processes outside their compartment is possible"; and Jail "effectively amounts to a virtual machine environment".

1.20 S20

- **Type:** Primary source
- **Credibility:** High — written by the author of FreeBSD jails about his own work, including an unflattering account of its predecessor.
- **Relevance:** Parts I and VII. The judgement of the author of FreeBSD jails that `chroot` was never a security mechanism.
- **URL:** <http://phk.freebsd.dk/sagas/jails/>

The span relied on: "chroot(2) was never intended to be a security mechanism nor a solid container, it was simply a name-space based hack". The page is written by Poul-Henning Kamp, the author of FreeBSD jails, so the span is a third party's assessment of `chroot` and **not** a statement by `chroot`'s own author, who is Bill Joy (V30). Every citation of this span names the author of FreeBSD jails as the speaker.

1.21 S21

- **Type:** Blog / editorial
- **Credibility:** Medium — an individual author's talk notes, self-flagged as second-hand on FreeBSD jails, but every kernel-version claim used here agrees with the man pages.
- **Relevance:** Parts VII and III. The 2002 mount-namespace date, the user-namespace completion trajectory, and why the user namespace is what makes rootless possible.
- **URL:** <https://k1024.org/papers/namespaces.html>

The spans relied on: the mount namespace arrived in kernel 2.4.19 as `CLONE_NEWNS`; `CLONE_NEWUSER` appeared in 2.6.23 and "the final pieces landed in 3.8"; and every namespace type other than the user namespace requires `CAP_SYS_ADMIN` to create.

1.22 S22

- **Type:** Primary source

- **Credibility:** High — a first-person account by a dotCloud and Docker engineer, self-described as retrospective, so its dates are recollection rather than record.
- **Relevance:** Parts VII and IV. What the first container engine actually was, and the later split of low-level container management into containerd.
- **URL:** <https://jpetazzo.github.io/2017/02/24/from-dotcloud-to-docker/>

The spans relied on: the engine began as a Python command-line tool that was "a front-end to LXC and AUFS"; LXC was replaced by the in-house `libcontainer` in 2014; and low-level container management was split out into containerd in Docker Engine 1.11.

1.23 S23

- **Type:** Reference
- **Credibility:** High — vendor documentation for the vendor's own build cache, and the source of the rule that generates all layer-ordering advice.
- **Relevance:** Parts II and III. The layer stack, one layer for each build instruction, and the downstream dependency between layers.
- **URL:** <https://docs.docker.com/build/cache/>

The spans relied on: "you can think of image layers as a stack, with each layer adding more content on top of the layers that came before it"; and "if a layer changes, all other layers that come after it are also affected".

1.24 S24

- **Type:** News
- **Credibility:** High — LWN's kernel reporting is technically rigorous and quotes the runc and LXC maintainers directly.
- **Relevance:** Part VI. The maintainers' position that a privileged container cannot be made secure.
- **URL:** <https://lwn.net/Articles/781013/>

The spans relied on: CVE-2019-5736 "once again, shows that privileged containers are difficult—perhaps impossible—to create in a secure manner"; the identification of Christian Brauner as "one of the LXC maintainers who worked on the runc fix", and his stronger form, that privileged containers "are simply never going to be truly secure"; and that `runc` is used by "Docker, Podman, Kubernetes, CRI-O, and containerd", which is the reach the escape had. The "difficult—perhaps impossible" wording is the LWN reporter's, not a maintainer's.

1.25 S25

- **Type:** Blog / editorial
- **Credibility:** High — a vendor engineering write-up of a public CVE with the mechanism shown step by step, corroborated by S24.
- **Relevance:** Part VI. How the first widely known escape worked, and the documented response of treating the kernel as single-tenant.
- **URL:** <https://aws.amazon.com/blogs/compute/anatomy-of-cve-2019-5736-a-runc-container-escape/>

The spans relied on: the entrypoint is set to a symlink to `/proc/self/exe` so that `runc` re-executes itself inside the container's filesystem; and the recommendation "to treat the Linux kernel as belonging to a single tenant, and spend your effort securing the kernel through another layer of separation... typically accomplished using a hypervisor or a virtual machine".

1.26 S26

- **Type:** Primary source
- **Credibility:** High — the runtime project's own security advisory, including its patch and end-of-life statements.
- **Relevance:** Part VI. The November 2025 escape class, and the fact that the fix landed in the runtime every engine shares.
- **URL:** <https://github.com/opencontainers/runc/security/advisories/GHSA-9493-h29p-rfm2>

The spans relied on: replacing the container's `/dev/null` with a symlink causes `runc` to bind-mount an attacker-chosen host path read-write; "as kernel upcalls are not namespaced, the configured binary... will run with full privileges on the host system"; the patches in `runc` 1.2.8, 1.3.3 and 1.4.0-rc.3; and the statement that 1.1.x and earlier were left unpatched.

1.27 S27

- **Type:** Blog / editorial
- **Credibility:** High — the foundation that hosts the runtime, writing a technical overview of advisories against its own project.

- **Relevance:** Part VI. An independent overview of the same 2025 class, the delivery path it names, and the state of the other two OCI runtimes at the time of the disclosure.
- **URL:** <https://www.cncf.io/blog/2025/11/28/runc-container-breakout-vulnerabilities-a-technical-overview/>

The spans relied on: the vulnerabilities "require the ability to start containers with custom mount configurations, making untrusted container images and Dockerfiles the **most likely** delivery method" — the hedge is the source's and the deck carries it; and, on the other implementations of the same runtime specification, "Other container runtimes like youki and crun are also **understood to** have similar flaws and are working on patches".

1.28 S28

- **Type:** Primary source
- **Credibility:** High — the project's own release notes, which state both the vulnerability class and the support window for each branch.
- **Relevance:** Part VI. The same `/dev` symlink class recurring after the 2025 fixes, which makes it a design seam rather than four separate bugs.
- **URL:** <https://github.com/opencontainers/runc/releases>

The spans relied on: CVE-2026-41579 "allowed a malicious image with a `/dev` symlink to have limited write access to the host filesystem"; and the notice that runc 1.3.z "will no longer be supported in less than 6 months (end of October 2026)".

1.29 S29

- **Type:** Academic paper
- **Credibility:** High — a systematic study across 11 runtimes with a stated method and a published report, and the only such study the deck relies on.
- **Relevance:** Part VI. The measured figures behind rootless being the highest-leverage mitigation and still incomplete.
- **URL:** <https://doi.org/10.2172/1890908>

The spans relied on: 59 CVEs across 11 container runtimes, of which 28 had public proof-of-concept exploits and 46% of those enabled container escape; the dominant root cause categorised as "host component leaked into the container"; and user namespaces preventing 7 of 9 tested escape exploits.

1.30 S30

- **Type:** Blog / editorial
- **Credibility:** High — a security-research write-up with a working proof of concept, describing a kernel bug rather than a runtime bug.
- **Relevance:** Part VI. The case no runtime fix could have prevented, which bounds what the five mechanisms promise.
- **URL:** <https://securitylabs.datadoghq.com/articles/dirty-pipe-container-escape-poc/>

The span relied on: Dirty Pipe (CVE-2022-0847) let an unprivileged process write to any file it could read, and was demonstrated escaping an unprivileged container by overwriting the page-cache copy of `runc`.

1.31 S31

- **Type:** Academic paper
- **Credibility:** High — a peer-reviewed study with a stated corpus and method. Its corpus is scientific data-analysis images rather than official images generally, and the figures it quotes for official images are taken from three earlier works, which the deck does not relay as this paper's own findings.
- **Relevance:** Part VI. What accumulates inside an image that nobody rebuilds, measured on a real corpus.
- **URL:** <https://academic.oup.com/gigascience/article/10/6/giab025/6291571>

The spans relied on: "container images used for neuroscience data analysis contain hundreds of vulnerabilities"; and that "software updates remove roughly two-thirds of these". The paper's introduction separately states, citing three other works rather than measuring it, that ">30% of official images in DockerHub have been shown to contain high-priority security vulnerabilities [7], images on average contain >180 vulnerabilities [8], and vulnerabilities are often caused by outdated packages [9]" — used in the deck only as a statement the paper reports, never as its own result.

1.32 S32

- **Type:** Blog / editorial
- **Credibility:** High — written in 2014 by the engineer who then led Red Hat's container security work, and its rule is still quoted by the projects he built.
- **Relevance:** Part VI. The operating rule that follows from the boundary being the syscall interface.

- **URL:** <https://opensource.com/business/14/7/docker-security-selinux>

The spans relied on: the title claim "Containers do not contain"; and the rule "Treat root within a container as if it is root outside of the container".

1.33 S33

- **Type:** Primary source
- **Credibility:** High — the project's own description of what it builds, which is the counter-example to an unscoped claim.
- **Relevance:** Part VI. A container runtime that builds virtual machines, which is why the deck scopes its claim to namespace-based Linux containers.
- **URL:** <https://katacontainers.io/>

The span relied on: Kata Containers is "An open source container runtime, building lightweight virtual machines that seamlessly plug into the containers ecosystem".

1.34 S34

- **Type:** Primary source
- **Credibility:** High — the normative runtime standard, which carries the virtual-machine section itself.
- **Relevance:** Part VI. The standard does not treat a VM-backed container as a deviation, so neither can the deck.
- **URL:** <https://github.com/opencontainers/runtime-spec/blob/main/config-vm.md>

The span relied on: the document "describes the schema for the virtual-machine-specific section of the container configuration".

1.35 S35

- **Type:** Reference
- **Credibility:** High — vendor documentation for the vendor's own kernel, describing the isolation mode it ships.
- **Relevance:** Part V. The one case where "a container on Windows runs in a Linux VM" does not apply, which is why the deck's claim names Linux containers.
- **URL:** <https://learn.microsoft.com/en-us/virtualization/windowscontainers/manage-containers/hyperv-container>

The span relied on: process-isolated Windows containers "share the same kernel with the host as well as each other. This is approximately the same as how Linux containers run".

1.36 S36

- **Type:** News
- **Credibility:** High — a vulnerability-research vendor reporting CVEs that carry vendor advisories and confirmed exploitation; the commercial interest is in the alert rather than in the advisory facts.
- **Relevance:** Part VI. A hypervisor escape exploited in the wild, which is why the deck hands the room no boolean at the virtual-machine boundary.
- **URL:** <https://www.tenable.com/blog/cve-2025-22224-cve-2025-22225-cve-2025-22226-zero-day-vulnerabilities-in-vmware-esxi>

The span relied on: for CVE-2025-22225, "an attacker with requisite privileges could exploit this vulnerability through the VMX process to escape the sandbox".

1.37 S37

- **Type:** Primary source
- **Credibility:** High — the vendor's own repository stating its own design.
- **Relevance:** Part V. A third macOS shape, one virtual machine per container, which differs from the single shared virtual machine the deck teaches for Lima.
- **URL:** <https://github.com/apple/containerization>

The span relied on: "Containerization executes each Linux container inside of its own lightweight virtual machine".

1.38 S38

- **Type:** Academic paper
- **Credibility:** High — a peer-reviewed NSDI paper in which the operator of a very large container platform publishes its own reversal, with the reason stated.
- **Relevance:** Parts I and VI. Part I: the microVM as the third answer beside a container and a full virtual machine, with the paper's own start-time and memory figures. Part VI: containers did not replace virtual machines; at that scale they went back inside them, for a structural reason.

- **URL:** <https://www.usenix.org/system/files/nsdi20-paper-agache.pdf>

The spans relied on: "When we first built AWS Lambda, we chose to use Linux containers to isolate functions... We were unsatisfied with this approach"; "their reliance on a single operating system kernel means that there is a fundamental tradeoff between security and code compatibility"; "We built Firecracker because we didn't want to choose"; the paper's own ceilings, stated under its minimal guest-kernel configuration — it "offers memory overhead of less than 5MB per container, boots to application code in less than 125ms" (megabytes, not mebibytes, and per container rather than per microVM; §5.2 separately measures around 3 MB of overhead per virtual machine); that a microVM gives each sandbox "its own virtual hardware, page tables, and operating system kernel"; that this "moves the security-critical interface from the OS boundary to a boundary supported in hardware and comparatively simpler software"; and that Firecracker is deployed "in two publically-available serverless compute services at Amazon Web Services (Lambda and Fargate), where it supports millions of production workloads".

1.39 S39

- **Type:** Reference
- **Credibility:** High — the project's own FAQ about its own removal decision.
- **Relevance:** Part VII. The execution plane was replaced under the format, which is the evidence for the distribution-plane argument.
- **URL:** <https://kubernetes.io/blog/2022/02/17/dockershim-faq/>

The span relied on: the "v1.24 release actually removed the dockershim".

1.40 S40

- **Type:** Reference
- **Credibility:** High — the project's own explanation, written to correct a misreading of the change in S39.
- **Relevance:** Part VII. The image format outlived the engine that invented it.
- **URL:** <https://kubernetes.io/blog/2020/12/02/dont-panic-kubernetes-and-docker/>

The span relied on: "The image that Docker produces isn't really a Docker-specific image—it's an OCI (Open Container Initiative) image".

1.41 S41

- **Type:** Blog / editorial
- **Credibility:** Medium — an individual engineer's investigation from 2014; the behaviour it documents is still reproducible and follows from the separation of control groups and namespaces in S1 and S2.
- **Relevance:** Parts III and VI. The counter-example any student can run in ten seconds, which is why the deck splits its two verbs.
- **URL:** <https://fabiokung.com/2014/03/13/memory-inside-linux-containers/>

The span relied on: "`/proc/meminfo`, `/proc/vmstat` and friends are not *containerized*... They will always display memory numbers from the host system".

1.42 S42

- **Type:** Standard / code
- **Credibility:** High — the engine's own source of truth for its default capability set, read directly rather than quoted from a comparison article.
- **Relevance:** Parts II and III. Podman's eleven default capabilities, and the absence of `CAP_SYS_PTRACE` from them.
- **URL:** <https://github.com/containers/common/blob/main/pkg/config/default.go>

The span relied on: `DefaultCapabilities` lists `CHOWN`, `DAC_OVERRIDE`, `FOWNER`, `FSETID`, `KILL`, `NET_BIND_SERVICE`, `SETFCAP`, `SETGID`, `SETPCAP`, `SETUID` and `SYS_CHROOT`.

1.43 S43

- **Type:** Standard / code
- **Credibility:** High — the other engine's own source of truth for the same list, so the comparison is source to source rather than blog to blog.
- **Relevance:** Parts II and III. Docker's fourteen default capabilities: the eleven in S42 plus `CAP_MKNOD`, `CAP_NET_RAW` and `CAP_AUDIT_WRITE`.
- **URL:** <https://github.com/moby/moby/blob/master/daemon/pkg/oci/caps/default.go>

The span relied on: the default set adds `CAP_MKNOD`, `CAP_NET_RAW` and `CAP_AUDIT_WRITE` to the eleven in S42.

1.44 S44

- **Type:** Blog / editorial
- **Credibility:** Medium — a foundation-hosted article reporting one author's measurements on one codebase, so the ratios are directional and are named as one measurement on the slide.
- **Relevance:** Part VI. What crossing the virtual-machine boundary costs for a bind mount, with the named fix.
- **URL:** <https://www.cncf.io/blog/2023/02/02/docker-on-macos-is-slow-and-how-to-fix-it/>

The spans relied on: macOS measured roughly 3.5 times slower than Linux with `VirtioFS` and about 10 times slower with `gRPC-FUSE`, dominated by many-small-file access in `node_modules`; and moving that directory into a named volume brought the timings back in line with Linux.

1.45 S45

- **Type:** Primary source
- **Credibility:** High — the vendor's own issue tracker, where the emulator's limitation is confirmed rather than inferred.
- **Relevance:** Part VI. Architecture mismatch on Apple Silicon fails with an unhelpful diagnostic rather than a clear one.
- **URL:** <https://github.com/docker/for-mac/issues/7137>

The span relied on: Rosetta does not implement AVX and is not expected to, so images built for CPUs with AVX fail with `Illegal instruction`.

1.46 S46

- **Type:** Reference
- **Credibility:** High — vendor documentation describing the vendor's own subsystem, in the exact words the deck is correcting.
- **Relevance:** Parts II and V. The Windows half of the room is told by its own documentation that it already runs a Linux kernel inside a lightweight virtual machine.
- **URL:** <https://learn.microsoft.com/en-us/windows/wsl/compare-versions>

The spans relied on: WSL 2 uses "a Linux kernel inside of a lightweight utility virtual machine (VM)"; and it "runs Linux distributions as isolated containers inside the managed VM".

1.47 S47

- **Type:** Reference
- **Credibility:** High — vendor documentation for the setting, with the version requirement stated.
- **Relevance:** Part II. What a Windows student checks before installing Podman inside WSL 2.
- **URL:** <https://learn.microsoft.com/en-us/windows/wsl/systemd>

The spans relied on: systemd is "now the default for the current version of Ubuntu that will be installed using the `wsl --install` command"; and the `[boot]` edit requires that "your WSL version is 0.67.6 or newer".

1.48 S48

- **Type:** Reference
- **Credibility:** High — the distribution's own release notes for the release most student machines run.
- **Relevance:** Part IV. The default that switches off the mechanism rootless containers are built on, on the commonest student platform.
- **URL:** <https://documentation.ubuntu.com/release-notes/24.04/>

The span relied on: "the Ubuntu kernel now restricts the use of unprivileged user namespaces" by default, through `kernel.apparmor_restrict_unprivileged_userns`.

1.49 S49

- **Type:** Reference
- **Credibility:** High — the registry operator's own published limits.
- **Relevance:** Part II. Why a lecture hall on one network address is not the same as one student pulling one image.
- **URL:** <https://docs.docker.com/docker-hub/usage/>

The span relied on: the limit for "Unauthenticated users | 100 per IPv4 address or IPv6 / 64 subnet" per six hours.

1.50 S50

- **Type:** Reference
- **Credibility:** High — the vendor's own licence terms for its own product.
- **Relevance:** Part II. The command-line-only rule in this course is a teaching decision rather than a licence constraint.
- **URL:** <https://docs.docker.com/subscription/desktop-license/>

The span relied on: the licence lists education among the uses for which Docker Desktop is free.

1.51 S51

- **Type:** Reference
- **Credibility:** High — the vendor's own per-distribution installation instructions, which are the supported path.
- **Relevance:** Part II. The Linux setup path for Docker Engine, and the image store that became the default in Docker 29.
- **URL:** <https://docs.docker.com/engine/install/>

The span relied on: the per-distribution repository setup steps, given per distribution rather than as one script.

1.52 S52

- **Type:** Reference
- **Credibility:** High — the project's own installation documentation.
- **Relevance:** Part II. The other Linux setup path, packaged in every major distribution.
- **URL:** <https://podman.io/docs/installation>

The span relied on: the per-distribution package install steps, and the "Built-in, no need to install" entries for Fedora CoreOS and Fedora Silverblue.

1.53 S53

- **Type:** Reference
- **Credibility:** High — the project's own template documentation, which also records the syntax change that dates every older tutorial.

- **Relevance:** Part II. The current macOS command and the template names.
- **URL:** <https://lima-vm.io/docs/templates/>

The spans relied on: "In Lima 1.x, template URLs included leading slashes, e.g. `template:///fedora`", with the current form being `limactl create template:fedora`; and the offered templates `docker`, `docker-rootful`, `podman` and `podman-rootful`.

1.54 S54

- **Type:** Reference
- **Credibility:** High — the project's own configuration reference, stating its own defaults.
- **Relevance:** Part II. The default mount is read-only, which is the failure a student reads as a container bug.
- **URL:** <https://lima-vm.io/docs/config/>

The spans relied on: the defaults "CPU: 4 cores", "Memory: 4 GiB", "Disk: 100 GiB" and "Mounts: ~ (read-only)".

1.55 S55

- **Type:** Reference
- **Credibility:** High — the project's own manual page for the command, stating its own default.
- **Relevance:** Part II. The other macOS path mounts the home directory read-write, so one command behaves differently on the two paths the deck offers.
- **URL:** <https://docs.podman.io/en/latest/markdown/podman-machine-init.1.html>

The span relied on: "Unless changed, the default values is `$HOME:$HOME`".

1.56 S56

- **Type:** Primary source
- **Credibility:** High — working code by the author of a standard reference on container security, which demonstrates the claim rather than asserting it.
- **Relevance:** Part III. A short program can create a container by issuing the same requests, which retires the idea that an engine is what makes one.
- **URL:** <https://github.com/lizrice/containers-from-scratch>

The span relied on: the repository creates a container from a short Go program that calls `clone` with namespace flags, changes the root directory, and writes to the control-group filesystem.

1.57 S57

- **Type:** Blog / editorial
- **Credibility:** Medium — a science-education researcher's summary of an established teaching sequence rather than a primary study, but it states the method and the condition under which it applies.
- **Relevance:** Every demonstration in Part II. Why the prediction is written before the observation and not after it.
- **URL:** <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

The spans relied on: the method suits "topics areas where students are likely to have strong intuitions which do not match scientific accounts of the world"; and predicting first gives "some sense of ownership in the following activity – a stake in the outcome as it were".

1.58 S58

- **Type:** Academic paper
- **Credibility:** High — the USENIX LISA '04 paper by the Solaris Zones designers, and the primary publication of the mechanism. The conference landing page returns HTTP 403 to automated fetches; the `legacy` path below serves the full text.
- **Relevance:** Parts I and VII. States the three needs the deck opens with, in the founding literature's own words, and dates Solaris Zones to November 2004.
- **URL:** https://www.usenix.org/legacy/event/lisa04/tech/full_papers/price/price.pdf

The spans relied on: "a system with multiple workloads needs to run those workloads in isolation, to ensure that applications can neither observe data from other applications nor affect their operation. It must also prevent applications from over-consuming system resources"; "two poorly written applications at a customer site both wanted to bind a network socket to port 80... the customer resolved the conflict by buying two servers!"; "Hard-coded log file locations or other pathnames can make it difficult to deploy two distinct versions of a particular application on the same node"; that virtual machines "suffer significant performance overheads. With either of these approaches, the cost of administering multiple operating system instances can be substantial"; and

"We have been gratified when casual users mistake the technology for a virtual machine". Bibliographic record: Price, D. and Tucker, A., "Solaris Zones: Operating System Support for Consolidating Commercial Workloads", USENIX LISA '04, November 2004.

1.59 S59

- **Type:** Blog / editorial
- **Credibility:** Medium — mechanically precise on `binfmt_misc` dispatch and QEMU block translation, but written by a continuous-integration vendor selling native ARM runners.
- **Relevance:** Part VI. `exec format error` as the symptom of an image built for the wrong architecture, and the `binfmt_misc` registration that decides whether a translator runs at all.
- **URL:** <https://www.warpbuild.com/guides/qemu-vs-native-arm64-builds>

The spans relied on: `exec format error` is the canonical symptom of an image built for the wrong architecture; under QEMU the dispatch mechanism is `binfmt_misc`, where a registered entry tells the kernel to hand a foreign-architecture binary to an interpreter rather than execute it directly; and that "syscalls follow a different path: the interpreter catches each one, converts the arm64 ABI arguments into the host ABI, and forwards it to the x64 kernel".

1.60 S60

- **Type:** Primary source
- **Credibility:** High — the normative standard for the image configuration, which is the document `podman image history` reads.
- **Relevance:** Part II, step 4. The image configuration's `history` array is what a layer listing prints, one entry per build instruction.
- **URL:** <https://github.com/opencontainers/image-spec/blob/main/config.md>

The spans relied on: the `history` field "Describes the history of each layer"; each entry's `created_by` is "The command which created the layer"; and an entry carries `empty_layer`, which marks a history item that created no filesystem change and therefore no layer.

1.61 S61

- **Type:** Reference

- **Credibility:** High — the engine's own manual page for the command this course runs, documenting its own defaults.
- **Relevance:** Parts II, III and VI. The default user-namespace mode, the resource-limit options that cgroups version 1 does not carry in rootless mode, and the option that selects an image by platform.
- **URL:** <https://docs.podman.io/en/latest/markdown/podman-run.1.html>

The spans relied on: "If `--userns` is not set, the default value is determined as follows... Otherwise, `--userns=host` is assumed"; `--memory`, `--cpus` and `--blkio-weight` each carry the note "This option is not supported on cgroups V1 rootless systems"; and `--platform=OS/ARCH` "Specify the platform for selecting the image... The `--platform` option can be used to override the current architecture and operating system".

1.62 S62

- **Type:** Academic paper
- **Credibility:** High — a peer-reviewed article in ACM SIGOPS Operating Systems Review by a researcher with no vendor interest. The failure rate it reports is a University of Arizona study's figure, quoted by this paper, and the deck attributes it that way.
- **Relevance:** Part I. The evidenced form of "it works on my machine": the barrier has a name in the literature and a measured failure rate.
- **URL:** https://polaris.imag.fr/arnaud.legrand/research/readings/acm_sigops_si_rsea/p71-boettiger.pdf

The spans relied on: the first barrier is headed "1. 'Dependency Hell'"; and "A recent study by researchers at the University of Arizona found that less than 50% of software could even be successfully built or installed [4]". That study's corpus is code published with computer-systems research papers.

1.63 S63

- **Type:** Academic paper
- **Credibility:** High — an IBM Research report published at ISPASS 2015, with the hardware, the workloads and the tuning stated. Its own headline finding limits what the deck may claim about performance, and its start-up figures are quoted from another benchmark rather than measured by it.

- **Relevance:** Part I. The start-up difference between a container and a virtual machine, reported by this paper, and the explicit statement that containers are not generally faster.
- **URL:** <https://www.read.seas.harvard.edu/~kohler/class/cs260r-s19/containerperf14.pdf>

The spans relied on: "Although this paper focuses on steady-state performance, other measurements [39] have shown that containers can start much faster than VMs (less than 1 second compared to 11 seconds on our hardware) because unlike VMs, containers do not need to boot another copy of the operating system" — the figures are reported by this paper and measured by its reference [39], and the deck says so; "Our results show that containers result in equal or better performance than VMs in almost all cases"; and "Both VMs and containers require tuning to support I/O-intensive applications".

1.64 S64

- **Type:** Reference
- **Credibility:** High — vendor documentation for the vendor's own storage model, stating the lifecycle rule the demonstration tests.
- **Relevance:** Part II, steps 2 and 3. What the writable layer is, when its contents disappear, and what a volume changes about that.
- **URL:** <https://docs.docker.com/engine/storage/>

The spans relied on: "By default all files created inside a container are stored on a writable container layer"; "Data written to the container layer doesn't persist when the container is destroyed"; and that volumes "retain data even after the containers using them are removed".

1.65 S65

- **Type:** Primary source
- **Credibility:** High — the project's own release notes for the release most students will install, stating its own removals.
- **Relevance:** Parts II and IV. Three removals that decide whether a student's machine works at all, and that date the older material on rootless networking.
- **URL:** <https://github.com/podman-container-tools/podman/releases/tag/v6.0.0>

The spans relied on: "Support for running on cgroups v1 systems has been removed"; "Support for the slirp4netns rootless network stack has been removed. Please use Pasta instead"; and "Support for running on Intel Macs has been removed".

1.66 S66

- **Type:** Reference
- **Credibility:** High — the project's own manual page for the command, stating what the command is and is not.
- **Relevance:** Part II, step 4. Podman ships no compose implementation of its own, which is the failure a student meets at that step.
- **URL:** <https://docs.podman.io/en/latest/markdown/podman-compose.1.html>

The span relied on: "podman compose is a thin wrapper around an external compose provider such as docker-compose or podman-compose".

1.67 S67

- **Type:** Reference
- **Credibility:** High — canonical kernel interface documentation for the call the tracing demonstration issues.
- **Relevance:** Parts II and III. The two sufficient conditions for attaching a tracer, which is why tracing a child differs from attaching to a stranger.
- **URL:** <https://man7.org/linux/man-pages/man2/ptrace.2.html>

The spans relied on: under the default Yama setting "the calling process must either have the CAP_SYS_PTRACE capability in the user namespace of the target process or it must have a predefined relationship with the target process. By default, the predefined relationship is that the target process must be a descendant of the caller"; and that the checks also depend on "any enabled Linux Security Module (LSM)—for example, SELinux, Yama, or Smack".

1.68 S68

- **Type:** Reference
- **Credibility:** High — vendor documentation conceding a limitation of the vendor's own product.
- **Relevance:** Parts II and III. The engine's own statement that a memory report inside a container shows the host.

- **URL:** https://docs.docker.com/engine/containers/resource_constraints/

The span relied on: "Inside the container, tools like `free` report the host's available swap, not what's available inside the container".

1.69 S69

- **Type:** Primary source
- **Credibility:** High — the project's own description of what it builds and why, at its canonical repository location.
- **Relevance:** Part III. A userspace filesystem exists whose stated purpose is the reading this deck argues against, which is evidence that the misconception is widely held.
- **URL:** <https://github.com/lxc/lxcfs>

The span relied on: LXCFS is "a small FUSE filesystem written with the intention of making Linux containers feel more like a virtual machine".

1.70 S70

- **Type:** Reference
- **Credibility:** High — the project's own specification of the runtime interface it implements.
- **Relevance:** Part IV. Which process owns the container on a current Docker installation, which the host process table shows and the received account does not.
- **URL:** <https://containerd.io/docs/2.3/runtime-v2/>

The spans relied on: "containerd, the daemon, does not directly launch containers"; and that each shim instance "communicates with containerd as a daemon, while parenting containers via invoking independent runtimes". The page's reparenting sentence — "If container uses the same PID namespace with shim process, its descendant processes will be reparented to shim process" — is conditional on a shared PID namespace and is not relied on for the deck's parentage claim.

1.71 S71

- **Type:** Reference
- **Credibility:** High — vendor documentation for the vendor's own configuration file, stating its own defaults.

- **Relevance:** Part II. The three Windows settings that decide whether the memory limit, the engine and the file ownership behave as the demonstration expects.
- **URL:** <https://learn.microsoft.com/en-us/windows/wsl/wsl-config>

The spans relied on: the `memory` default is "50% of total memory on Windows"; the `[automount] cgroups` key "Selects the cgroup hierarchy to mount in WSL 2. Available values are `v1` and `v2`"; and `metadata`, "Whether metadata is added to Windows files to support Linux system permissions", is `disabled` by default.

1.72 S72

- **Type:** Reference
- **Credibility:** High — vendor documentation describing the performance property of the vendor's own file-sharing path.
- **Relevance:** Parts II and VI. Where a Windows student should keep working files, and why the ownership demonstration cannot be run from a Windows drive.
- **URL:** <https://learn.microsoft.com/en-us/windows/wsl/filesystems>

The span relied on: "your performance speed will improve if you store them directly on the `\\wsl$` drive".

1.73 S73

- **Type:** Reference
- **Credibility:** High — the project's own security documentation, written by the group with the largest production exposure to the question.
- **Relevance:** Part VI. The comparison stated by a neutral project rather than by a vendor of either side.
- **URL:** <https://kubernetes.io/docs/concepts/security/multi-tenancy/>

The spans relied on: containers "offer a weaker isolation boundary than virtual machines that utilize hardware-based virtualization"; and "Sandboxing is often recommended when you are running untrusted code, where workloads are assumed to be malicious".

1.74 S74

- **Type:** Reference

- **Credibility:** High — the kernel's own documentation for the filesystem, which is the layer the engine's storage driver drives.
- **Relevance:** Part III. The kernel names and owns the mechanism a storage driver only configures.
- **URL:** <https://docs.kernel.org/filesystems/overlayfs.html>

The span relied on: "An overlay-filesystem tries to present a filesystem which is the result of overlaying one filesystem on top of the other".

1.75 S75

- **Type:** Reference
- **Credibility:** High — vendor documentation that states the mechanism plainly, and contradicts the same vendor's marketing page.
- **Relevance:** Parts I and VI. The engine's own teaching documentation already agrees with the thesis, which shows the disagreement is with the sales copy rather than with the field.
- **URL:** <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

The span relied on: "Simply put, containers are isolated processes for each of your app's components".

1.76 S76

- **Type:** Reference
- **Credibility:** High — vendor documentation for the vendor's own feature, in the exact words that manufacture the misconception the deck corrects.
- **Relevance:** Parts II and III. A volume is the engine's bookkeeping, and the engine says so in words that sound like a kernel guarantee.
- **URL:** <https://docs.docker.com/engine/storage/volumes/>

The spans relied on: "Volumes are completely managed by Docker and are isolated from the core functionality of the host machine"; "Volumes are the preferred mechanism for persisting data generated by and used by Docker containers"; and "Volumes are easier to back up or migrate than bind mounts".

1.77 S77

- **Type:** Blog / editorial
- **Credibility:** High — the distribution vendor announcing its own kernel default, with the exact policy rule quoted.
- **Relevance:** Part IV. The host policy that switches off the mechanism rootless containers rest on, stated at the point it was introduced.
- **URL:** <https://ubuntu.com/blog/ubuntu-23-10-restricted-unprivileged-user-namespaces>

The span relied on: unprivileged processes "will only be able to create user namespaces if they are confined and have the 'usersns,' rule in their AppArmor profile (or if they have CAP_SYS_ADMIN)".

1.78 S78

- **Type:** Reference
- **Credibility:** High — the configuration file's own manual page, which is where the installed default is read from.
- **Relevance:** Part IV. The current rootless network backend, which replaced the one the 2020 account describes.
- **URL:** <https://github.com/containers/common/blob/main/docs/containers.conf.5.md>

The span relied on: the shipped setting `"default_rootless_network_cmd"="pasta"`.

1.79 S79

- **Type:** Reference
- **Credibility:** High — the vendor's own troubleshooting documentation for its own product, naming the error code and the fix.
- **Relevance:** Part II. The commonest Windows setup failure, and the firmware setting that causes it.
- **URL:** <https://learn.microsoft.com/en-us/windows/wsl/troubleshooting>

The spans relied on: "Installation failed with error 0x80070003 or error 0x80370102 — Please make sure that virtualization is enabled inside of your computer's BIOS"; and "Please enable the Virtual Machine Platform Windows feature and ensure virtualization is enabled in the BIOS".

1.80 S80

- **Type:** Reference
- **Credibility:** High — the vendor's own documentation for its own feature, stating its costs.
- **Relevance:** Part II, step 3. Why a bind mount is not enough: it ties the container to one host's directory layout, and it lets the container change host files.
- **URL:** <https://docs.docker.com/engine/storage/bind-mounts/>

The spans relied on: "Containers with bind mounts are strongly tied to the host. Bind mounts rely on the host machine's filesystem having a specific directory structure available"; and "you can change the host filesystem via processes running in a container, including creating, modifying, or deleting important system files or directories".

1.81 S81

- **Type:** Reference
- **Credibility:** High — the standard itself, which every conforming OCI runtime reads.
- **Relevance:** Part III. The two configuration fields that sit outside the Linux-specific section: `root`, the assembled root filesystem, and `process.capabilities`, the capability sets the container process starts with.
- **URL:** <https://github.com/opencontainers/runtime-spec/blob/main/config.md>

The spans relied on: the `root` object, "specifies the container's root filesystem", whose `path` is "the absolute path or path relative to the bundle"; and `process.capabilities`, "an object containing arrays that specify the sets of capabilities for the process", with its `bounding`, `effective`, `inheritable`, `permitted` and `ambient` arrays.

1.82 S82

- **Type:** Reference
- **Credibility:** High — the vendor's own supported installation procedure for the distribution this course's Windows path installs.
- **Relevance:** Part II. Why a Linux student does not simply `apt install docker.io`: Docker's own procedure removes the distribution's packages first, because they conflict with Docker's own.
- **URL:** <https://docs.docker.com/engine/install/ubuntu/>

The spans relied on: the "Uninstall conflicting packages" step, which states that "before you can install Docker Engine, you need to uninstall any conflicting packages" and names `docker.io`, `docker-compose`, `docker-compose-v2`, `docker-doc`, `docker-buildx` and `podman-docker` as packages "which may conflict with the official packages provided by Docker" — the page hedges with *may*, and the deck carries that hedge; and the page's own "Install using the convenience script" route, fetched from `get.docker.com`, which is the route the deck's Linux and Windows columns use because the full repository-setup block does not fit on a slide.

1.83 S83

- **Type:** Reference
- **Credibility:** High — the project's own home page, which is where its install command is published.
- **Relevance:** Part II. What Homebrew is and the one command that installs it, which every macOS student runs before `brew install lima`.
- **URL:** <https://brew.sh>

The spans relied on: the page's own title, "Homebrew: The Package Manager for Everywhere", and its answer to "What Does Homebrew Do?" — "Homebrew installs the command-line tools and applications you need across macOS, Linux and WSL"; the published install command `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`, which the page offers as one of several routes beside a macOS `.pkg` installer; and the documented install prefixes, `/opt/homebrew` on Apple Silicon, `/home/linuxbrew/linuxbrew` on Linux/WSL and `/usr/local` on macOS Intel. The page's own wording changed before 2026-09-29, and the older "Missing Package Manager" self-description is no longer on it.

1.84 S84

- **Type:** Reference
- **Credibility:** High — the kernel's own documentation for the module that applies the tracing restriction.
- **Relevance:** Part III and Part IV. What Yama is in plain words, and what each `ptrace_scope` level allows — the definition the deck owed the room before it read a `ptrace` refusal, and the reason tracing Docker's daemon tree needs `sudo`.
- **URL:** <https://docs.kernel.org/admin-guide/LSM/Yama.html>

The spans relied on: that Yama is a Linux Security Module collecting system-wide DAC security protections, and the four `kernel.yama.ptrace_scope` levels — 0, "classic ptrace permissions"; 1, "restricted ptrace", where a process may `PTRACE_ATTACH` only to a descendant or to a process that declared it a tracer, unless the tracer holds `CAP_SYS_PTRACE`; 2, "admin-only attach", where only a process with `CAP_SYS_PTRACE` may attach; and 3, "no attach", which cannot be lowered without a reboot.

1.85 S85

- **Type:** Reference
- **Credibility:** High — the vendor's own documented post-installation procedure.
- **Relevance:** Part II. The step that lets a student's own account use the Docker daemon socket, without which the first `docker run` on a freshly installed Linux or WSL engine fails with a permission error.
- **URL:** <https://docs.docker.com/engine/install/linux-postinstall/>

The spans relied on: the "Manage Docker as a non-root user" procedure, `sudo usermod -aG docker $USER` followed by `newgrp docker`, and its statement that the Docker daemon binds to a UNIX socket owned by root, so a non-root account must join the `docker` group to reach it.

1.86 S86

- **Type:** Reference
- **Credibility:** High — the project's own installation documentation, which is where the PATH step is written down.
- **Relevance:** Part II. Why a macOS student must run the lines the installer prints before `brew` works at all, which `brew.sh` itself does not state.
- **URL:** <https://docs.brew.sh/Installation>

The span relied on: "When you install Homebrew, it prints some directions for updating your shell's config. If you don't follow those directions, Homebrew will not work."

2 Vault sources

2.1 V1 — Container Isolation and Distribution via Docker and Podman

This note argues that a container is not a kernel object but an ordinary process composed from five independent kernel mechanisms — namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it keeps), seccomp (which syscalls it may issue), and a union filesystem (what it sees as its root). Each mechanism is documented separately, sourced to the Linux manual pages and the OCI image and runtime specifications, and the note traces their history as parallel, only loosely connected developments — `chroot`, FreeBSD jails, Solaris Zones, and Linux namespaces and cgroups arose from shared-hosting pressure rather than a single inherited design. It credits Docker's real innovation as the separation of a distribution plane (a layered, content-addressed image format plus a registry protocol) from the execution plane, which the kernel primitives already provided, and traces Docker's own origin from dotCloud's failing PaaS and its 2013 open-sourcing. Podman is presented as an architectural rebuttal to Docker rather than a rejection of its kernel usage: it keeps the OCI image format and a Docker-compatible CLI but replaces the root-owned background daemon with a per-invocation fork-exec model, monitored by a small helper process, and is built around rootless operation via user namespaces from the start.

The note is explicit about limits. Isolation is bounded by the entire Linux system-call surface, which is far larger than a hypervisor boundary, and it documents this with specific runtime-escape CVEs (including one that let a malicious image overwrite the host container runtime binary, and a 2025 set exploiting mount-path handling) plus a peer-reviewed study finding that user namespaces prevented most tested escapes. It flags several claims as weaker than the rest: a runtime benchmark comparing two OCI runtimes is marked as self-reported by the project that ran it, and a widely repeated vulnerability-percentage figure for container images is marked unverified because it comes from vendors with a commercial interest in the number. The note also states open questions it did not resolve — the precise kernel version at which several individual namespace types were introduced, and, as of its most recent correction pass, which userspace network backend is now the default for rootless Podman versus an older maintainer account it still quotes as historical. It explicitly rejects one frequently repeated comparison claim (a specific capability-count difference between the two engines) as unsupported by any primary source it could find, rather than including it uncritically.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**

- [Namespaces](#) — Justin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — GigaScience (2021), stable DOI alternative to the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**

- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.2 V2 — Docker and Podman Containers

This note treats a container as an ordinary Linux process, not a kernel object, held inside an illusion built from five independent, separately-configured mechanisms: namespaces, cgroups, capabilities, seccomp, and a union-mounted root filesystem. Most of what containers get wrong follows from where that illusion meets the real machine underneath it. The note catalogues specific seams where this shows: OverlayFS's `copy_up` operation copies an entire file on the first write, however small the change, because OverlayFS works at file rather than block granularity, so appending one byte to a multi-gigabyte log copies the whole file. `strace` fails inside a default container not from an active block but because `CAP_SYS_PTRACE` sits outside the default capability set, and seccomp additionally blocked `ptrace` on kernels before 4.8. And off Linux there is no native container at all — "Docker on a Mac" is Docker inside a Linux VM — which produces measured bind-mount slowdowns over VirtioFS and gRPC-FUSE, and silent architecture-mismatch failures such as `Illegal instruction` under Rosetta or `exec format error` under QEMU.

The note does not treat these seams as defects to patch: it presents them as the necessary cost of a shared kernel, and states outright that some failures — Dirty Pipe escaping a container by overwriting the page-cache copy of `runc` — bypass the container runtime entirely, so no runtime fix could have stopped them. Several supporting figures are explicitly flagged as directional rather than verified: crun's speed comparison

against runc is the crun project's own self-reported benchmark, and the macOS bind-mount slowdown multipliers and the claimed hour-versus-minutes emulated-build penalty rest on single-author measurements of one project's shape, not controlled studies. A widely repeated claim comparing Podman's and Docker's default capability counts is deliberately left out, because every instance traced back to uncited blogs rather than the projects' own defaults. The note also leaves open which network backend a student's actual Podman release defaults to, and records rather than resolves a one-release disagreement between two of Podman's own docs pages over the kernel version needed for native rootless overlay storage.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**

- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — DOI stable alternative for the source above (publisher URL returns HTTP 403 to datacenter clients); GigaScience (2021) **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**

- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCf org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.3 V3 — Containers with Docker and Podman

This note argues that a container is not a kernel object but an ordinary process subjected to five independently-configurable kernel mechanisms — namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it keeps), seccomp (which syscalls it may issue), and a union-mounted filesystem (what it sees as its root) — and that Docker and Podman are simply the programs that issue those kernel requests in a repeatable order and package the filesystem the process wakes up in. It traces each mechanism to primary kernel documentation and dates its introduction, and it complicates the standard `chroot` → jails → Zones → LXC → Docker lineage: Linux namespaces, BSD jails and Solaris Zones are presented as parallel responses to the same shared-hosting pressure rather than a single inherited design, while cgroups have a separately documented origin at Google in 2006. It further argues that Docker's real

contribution over the pre-existing LXC was distribution — a layered, content-addressed image format plus a registry protocol — not isolation, which already existed, and that Podman is best read as a rejection of Docker's daemon architecture rather than its kernel usage: both use the same namespaces, cgroups and OCI-standardised image and runtime formats, differing chiefly in whether a privileged background daemon or a per-invocation fork-exec process owns the container.

The note does not claim containers match hypervisor-grade isolation: it states that the isolation boundary is the entire Linux system-call surface, cites several `runc` container-escape and file-descriptor-leak CVEs plus a peer-reviewed study of 59 CVEs across 11 container runtimes, and treats it as settled that privileged containers cannot be made secure. Several points are explicitly left unsettled rather than asserted: the note flags a genuine disagreement between its own sources over `chroot`'s origin date and lineage; it records, without resolving, a one-release discrepancy between Podman's own documentation pages over the kernel version needed for native rootless overlay storage; and it marks a `crun`-vs-`runc` benchmark, an emulated-build slowdown figure, macOS bind-mount timings, and a rootless-networking throughput measurement as self-reported, vendor-supplied, or single-study results rather than independently replicated ones. A closing "Grounding gap" section lists claims the author deliberately excluded for lacking primary sourcing — notably a widely repeated capability-count comparison between Podman and Docker traced to uncited SEO blogs — and names Podman's current default rootless network backend as a question the note raises but does not settle.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**

- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\)](#) — Podman manual page — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman](#) — project README — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun](#) — project README — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**

- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025 \(DOI, stable alternative locator\)](#) — GigaScience (2021) **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.4 V4 — Docker and Podman as implementations of Linux container primitives

This note argues that a Linux container is not a kernel object but an ordinary process subjected to five independently configurable kernel mechanisms — namespaces, cgroups, capabilities, seccomp filtering, and a union-mounted root filesystem — and that removing any one of the five leaves something that still runs but is less of a container. It traces the lineage of these mechanisms (`chroot`, BSD jails, Solaris Zones, Linux namespaces and cgroups) as parallel developments rather than a single inherited chain, drawing on primary interviews, kernel commit history and man-page text, and situates Docker's real innovation as the layered, content-addressed image format and registry protocol rather than the underlying isolation primitives, which already existed via LXC. It then contrasts Docker's client-daemon-containerd-runc architecture with Podman's daemonless fork-exec model built around `libpod`, `conmon` and `crun`, showing that both engines use the same OCI image format, the same kernel mechanisms, and interchangeable OCI runtimes — so the substantive disagreement between them is about which process owns the container's lifecycle: a long-lived, root-owned daemon versus a process tree rooted directly in the user's own shell or a systemd unit.

The note is explicit about what it does not claim. It states that shared-kernel isolation is weaker than a hypervisor boundary, documenting several `runc` CVEs (2019-5736, 2024-21626, and a November 2025 `maskedPaths / core_pattern` set) plus a kernel-level Dirty Pipe escape that no runtime fix could have prevented, and it treats "privileged containers cannot be made secure" as settled rather than debatable. Several points are flagged as open rather than resolved: which rootless network backend (`slirp4netns`, `pasta`, or Netavark) ships as default in a given Podman release was tracked as unsettled and only pinned, as of 2026-09-29, for Podman v6.1.2; a widely repeated claim that Docker and Podman ship different default kernel-capability counts is deliberately excluded from the note as untraceable to a primary source; and exact per-namespace kernel-introduction versions are noted as resting on secondary talk notes rather than the current man page. Several cited figures — `crun` vs `runc` benchmark timings, macOS bind-mount slowdown, and ARM64 emulation cost — are explicitly marked as vendor- or single-author-reported rather than independently verified.

Referenced sources

- [namespaces\(7\)](#) — Linux manual page, Michael Kerrisk / The Linux man-pages project — **HIGH**
- [cgroups\(7\)](#) — Linux manual page, Michael Kerrisk / The Linux man-pages project — **HIGH**

- [capabilities\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project — **HIGH**
- [seccomp\(2\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project — **HIGH**
- [OCI Image Format Specification](#), Open Container Initiative — **HIGH**
- [OCI Runtime Specification](#), Open Container Initiative — **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#), Open Container Initiative — **HIGH**
- [Seccomp security profiles for Docker](#), Docker Inc. — **HIGH**
- [Use the OverlayFS storage driver](#), Docker Inc. — **HIGH**
- [Docker overview](#), Docker Inc. — **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#), Docker Inc. — **HIGH**
- [Docker build cache](#), Docker Inc. — **HIGH**
- [podman\(1\) — Podman manual page](#), The Podman project / Podman Container Tools (CNCF) — **HIGH**
- [podman-container-tools/podman — project README](#), The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [containers/crun — project README](#), The crun project / containers organisation — **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#), Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) — **HIGH**
- [What is Podman?](#), Red Hat (2026) — **MEDIUM**
- [Why strace doesn't work in Docker](#), Julia Evans (2020) — **HIGH**
- [From dotCloud to Docker](#), Jérôme Petazzoni (2017) — **HIGH**
- [Docker grows to harbor innovation](#), Alex Handy / SD Times (2014) — **MEDIUM**
- [A Pivot that Worked: The Docker Story](#), Dan Scholnick / Trinity Ventures (2019) — **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#), Stefano Noferi (2013) — **MEDIUM**
- [History of Containers](#), Antranig Vartanian — **MEDIUM**
- [Jails – High value but shitty Virtualization](#), Poul-Henning Kamp — **HIGH**
- [Namespaces](#), Iustin Pop (2019) — **MEDIUM**
- [A container-confinement breakout](#), Jake Edge / LWN.net (2019) — **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#), Amazon Web Services (2019) — **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#), opencontainers/runc maintainers — **HIGH**

- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#), Sysdig (2025) — **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#), Cloud Native Computing Foundation (2025) — **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#), Red Hat Product Security — **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#), Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) — **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#), Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) — **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#), GigaScience (2021) — **HIGH**
- [10.1093/gigascience/giab025](#) (stable alternative locator for the above), GigaScience (2021) — **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#), arXiv preprint — **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#), WarpBuild — **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#), Syl Taylor (2023) — **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#), Docker for Mac issue tracker — **MEDIUM**
- [Docker on macOS is slow and how to fix it](#), Paolo Mainardi / CNCF (2023) — **MEDIUM**
- [Lima — Linux virtual machines](#), The Lima project (CNCF sandbox) — **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#), Abiola Ibrahim / the Colima project — **HIGH**
- [Get started with Docker remote containers on WSL 2](#), Microsoft Learn — **HIGH**
- [Install Docker Engine](#), Docker Inc. — **HIGH**
- [Podman Installation Instructions](#), The Podman project — **HIGH**
- [podman-machine\(1\)](#), The Podman project — **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#), Ivan Velichko — **MEDIUM**
- [multiarch/qemu-user-static](#), The multiarch project — **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11, Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) — **HIGH**

- [Podman is gaining rootless overlay support](#), Daniel Walsh / Red Hat — Enable Sysadmin (2021) — **HIGH**
- [podman/rootless.md](#), The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [podman/docs/tutorials/performance.md](#), The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [WSL2-Linux-Kernel releases](#), Microsoft (2024–2026) — **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#), Naoki Matsumoto and Akihiro Suda (2024) — **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699, Podman core maintainers (2026) — **HIGH**

2.5 V5 — Linux containers and the Docker/Podman engines built on them

This note argues that a container is not a kernel object at all but an ordinary process that several independent, pre-existing Linux interfaces — namespaces, cgroups, capabilities, seccomp, and a union-mounted root filesystem — have been asked to constrain and disguise. It traces the underlying isolation primitives back through `chroot` (1982), FreeBSD jails (1999–2000) and Solaris Zones, and separately to the 2002 mount namespace and the 2008 cgroups merge, arguing — citing the kernel man pages, the OCI Image and Runtime specifications, and first-person accounts from Docker's and Podman's own engineers — that these lineages are parallel responses to shared-hosting pressure rather than one inherited design. Its central claim about Docker specifically is that isolation was already solved before Docker existed; Docker's actual contribution was the distribution plane — an immutable, content-addressed, layered image format plus a registry transport — built out of a failing PaaS company's earlier LXC/AUFS tooling. Podman is presented as a challenge to Docker's daemon architecture rather than to its kernel usage: the same OCI images and largely the same primitives, run daemonless and rootless by default. The note also documents several concrete runc container-escape CVEs and a peer-reviewed study finding that user namespaces block most tested escapes, as support for preferring rootless operation.

The note is explicit about what it does not establish. Several claims are flagged inline as inference, self-reported, or unverified rather than settled fact: the `crun`-vs-`runc` benchmark numbers are the `crun` project's own figures on its own benchmark; the reported macOS emulation and bind-mount slowdown multipliers come from single-author, uncontrolled measurements; and a claim that vendor scanners put 64–90% of popular images at high vulnerability risk is explicitly logged as unverified and commercially sourced. A "Grounding gap" section lists claims it deliberately excluded for lacking a

primary source — notably a widely repeated but unsourced figure comparing Podman's and Docker's default capability counts — and leaves open exactly which rootless network backend ships by default in the Podman release students will install, though a later correction narrows this using the project's own installation docs. The note also does not claim containers match hypervisor-grade isolation: it states plainly that the shared kernel's entire system-call surface remains the attack surface, and treats that boundary, not any single exploit, as the unresolved question.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**

- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis \(DOI\)](#) — GigaScience (2021), stable alternative locator **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**

- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.6 V6 — Containers as an Assembly of Independent Kernel Mechanisms

The note's central claim is that a container is not a kernel object at all: there is no `struct container` and no single subsystem that owns the concept. Instead, a container is an ordinary process that the kernel has been asked, through five separately-developed and independently-configurable interfaces, to lie to about what it can see and consume — namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it keeps), seccomp (which syscalls it may issue), and a union filesystem (what it sees as `/`). Removing any one of the five still leaves something that runs, just with less isolation. The note backs this with primary kernel documentation (the `namespaces(7)`, `cgroups(7)`, `capabilities(7)` and `seccomp(2)` man pages) and the OCI image and runtime specifications, and traces the historical lineage — `chroot` (1982), FreeBSD jails (1999), Solaris Zones, Linux namespaces (2002) and cgroups (2008), LXC, and Docker's 2013 open-sourcing out of the failing dotCloud PaaS — using first-hand en-

gineer accounts and contemporaneous reporting. Its account of Docker's real contribution is that it separated the execution plane (kernel primitives, already solved) from the distribution plane (a layered, content-addressed image format plus a transport protocol), and that Podman's daemonless, fork-exec architecture is an argument against Docker's *architecture* rather than its kernel usage, evidenced by the project's own manual pages and a Red Hat maintainer walkthrough of a rootless container start.

The note is explicit about what it does not claim. It does not claim container isolation is comparable to a hypervisor's: sharing a kernel means the attack surface is the entire syscall interface, and it treats "privileged containers are simply never going to be truly secure" as settled, citing LWN and a runc maintainer. Several points are left openly unsettled rather than resolved: Podman's own two documentation pages give different kernel-version thresholds (5.12 vs. 5.13) for native rootless overlay support, and the note records that disagreement instead of picking one. A recorded "Grounding gap" section excludes a frequently-repeated claim about a specific capability-count difference between Podman and Docker because every source traced back to uncited SEO blogs, and flags vendor-reported vulnerability-prevalence figures (64–90% of images) and a runtime benchmark as self-reported and not independently corroborated. The note also notes it could not pin exact per-namespace kernel introduction versions because the current man page dropped its version column, resting that claim on secondary talk notes instead.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — HIGH
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — HIGH
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — HIGH
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — HIGH
- [OCI Image Format Specification](#) — Open Container Initiative — HIGH
- [OCI Runtime Specification](#) — Open Container Initiative — HIGH
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative — HIGH
- [Seccomp security profiles for Docker](#) — Docker Inc. — HIGH
- [Use the OverlayFS storage driver](#) — Docker Inc. — HIGH
- [Docker overview](#) — Docker Inc. — HIGH
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. — HIGH

- [Docker build cache](#) — Docker Inc. — **HIGH**
- [podman\(1\)](#) — Podman manual page — The Podman project / Podman Container Tools (CNCF) — **HIGH**
- [podman-container-tools/podman](#) — project README — The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [containers/crun](#) — project README — The crun project / containers organisation — **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) — **HIGH**
- [What is Podman?](#) — Red Hat (2026) — **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) — **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) — **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) — **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) — **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) — **MEDIUM**
- [History of Containers](#) — Antranig Vartanian — **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp — **HIGH**
- [Namespaces](#) — Justin Pop (2019) — **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) — **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) — **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers — **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) — **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) — **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security — **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) — **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) — **HIGH**

- [An analysis of security vulnerabilities in container images for scientific data analysis \(stable alternative locator: DOI 10.1093/gigascience/giab025\)](#) — GigaScience (2021) — **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint — **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild — **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) — **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker — **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) — **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) — **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project — **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn — **HIGH**
- [Install Docker Engine](#) — Docker Inc. — **HIGH**
- [Podman Installation Instructions](#) — The Podman project — **HIGH**
- [podman-machine\(1\)](#) — The Podman project — **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko — **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project — **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) — **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) — **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCf) — **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCf) — **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) — **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) — **MEDIUM**
- [CNCf org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) — **HIGH**

2.7 V7 — Containers as an Assembly of Independent Kernel Mechanisms

The note argues that a container is not a kernel object — there is no `struct container`, no `create_container()` syscall, and no single subsystem that owns the concept — but an ordinary process that five independently-developed kernel interfaces have each been asked to constrain: namespaces restrict what the process can see, cgroups restrict what it can consume, capabilities restrict how much of root it keeps, seccomp restricts which syscalls it may issue, and a union filesystem determines what it sees as `/`. Removing any one of the five still leaves something that runs, just with less isolation, and the five are configured independently of one another. It backs each mechanism with the primary kernel documentation (the `namespaces(7)`, `cgroups(7)`, `capabilities(7)` and `seccomp(2)` man pages) and the OCI image and runtime specifications, and traces the historical lineage carefully rather than repeating the usual clean `chroot` → jails → Zones → LXC → Docker chain: `chroot` (BSD, 1982), FreeBSD jails (1999) and Solaris Zones are presented as parallel responses to a shared-hosting problem rather than a single inherited design, since Linux namespaces (2002) and cgroups (2008) trace instead to Plan 9 and to Google's "process containers" work respectively. Docker's own history is drawn from first-hand engineer accounts of the failing dotCloud PaaS, and its real contribution is framed as separating the execution plane (kernel primitives, already solved) from the distribution plane (a layered, content-addressed image format plus a transport protocol) — with Podman's daemonless, fork-exec architecture presented as an argument against Docker's *architecture* rather than its kernel usage, evidenced by the project's own manual pages and a Red Hat maintainer's walkthrough of a rootless container start.

The note is explicit about what it does not claim. It does not treat container isolation as comparable to a hypervisor's, since sharing a kernel exposes the entire syscall interface as attack surface, and it treats the position that "privileged containers are simply never going to be truly secure" as settled, citing LWN and a runc maintainer rather than asserting it independently. Several points are left openly unsettled: Podman's own two documentation pages give different kernel-version thresholds (5.12 vs. 5.13) for native rootless overlay support, and the note records that disagreement rather than picking one silently. A dedicated "Grounding gap" section excludes a frequently-repeated claim about a specific capability-count difference between Podman and Docker because every source traced back to uncited SEO blogs, and it separately flags vendor-reported vulnerability-prevalence figures (64–90% of images) and a runtime benchmark as self-reported figures not independently corroborated. The note also notes it could not pin exact per-namespace kernel introduction versions, since the current man page dropped its version column, and rests that claim on secondary sources instead.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative — **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative — **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative — **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. — **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. — **HIGH**
- [Docker overview](#) — Docker Inc. — **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. — **HIGH**
- [Docker build cache](#) — Docker Inc. — **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) — **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation — **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) — **HIGH**
- [What is Podman?](#) — Red Hat (2026) — **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) — **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) — **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) — **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) — **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) — **MEDIUM**
- [History of Containers](#) — Antranig Vartanian — **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp — **HIGH**

- [Namespaces](#) — Justin Pop (2019) — **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) — **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) — **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers — **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) — **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) — **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security — **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) — **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) — **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) (stable alternative locator: DOI 10.1093/gigascience/giab025) — GigaScience (2021) — **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint — **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild — **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) — **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker — **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) — **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) — **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project — **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn — **HIGH**
- [Install Docker Engine](#) — Docker Inc. — **HIGH**
- [Podman Installation Instructions](#) — The Podman project — **HIGH**
- [podman-machine\(1\)](#) — The Podman project — **HIGH**

- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko — **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project — **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) — **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) — **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) — **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) — **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) — **HIGH**

2.8 V8 — The Kernel Mechanisms and Security Trade-offs of Docker and Podman

This note's central claim is that a container is not a kernel object but an ordinary process subjected to five independent, separately-configurable kernel mechanisms: namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it keeps), seccomp (which syscalls it may issue), and a union filesystem (what it sees as `/`). Each mechanism is backed by its own kernel manual page or the OCI specifications, and the note traces the history behind them — `chroot` in 1982, FreeBSD jails in 1999–2000, Solaris Zones, and the 2002/2008 arrival of Linux namespaces and cgroups — arguing from primary and first-person sources (including the jails author and a dotCloud/Docker engineer) that these are largely parallel lineages rather than one inherited chain, and that Docker's real innovation was the layered, content-addressed image format and registry protocol, not the isolation primitives themselves. It contrasts Docker's daemon-based architecture with Podman's daemonless fork-exec model, and covers documented rootless-mode mechanics, OverlayFS behavior, and OCI standardization on evidence from vendor documentation, kernel commits, and a peer-reviewed CVE study.

The note is explicit about what remains unsettled. It does not claim container isolation matches a hypervisor's — the shared kernel is stated as the entire attack surface, illustrated with named CVEs (runc escapes, Dirty Pipe) rather than asserted abstractly. Several figures are flagged by the note's own convention as inference, reported, or unverified: emulated-build slowdown magnitudes, macOS bind-mount benchmarks, and vendor claims of 64–90% vulnerable images are marked as uncorroborated or commercially motivated rather than settled fact. A widely repeated comparison ("Podman's rootless containers get 11 kernel capabilities compared to Docker's 14") is deliberately excluded as untraceable to any primary source. The note also records open questions it did not resolve, including the exact kernel version at which each namespace type was introduced, and logs dated corrections it has since applied — for example, walking back a superseded 2020 claim that the kernel forbids rootless overlay mounts. Throughout, the note treats itself as a living, periodically corrected document rather than a finished one.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**

- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — stable DOI alternative for the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**

- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.9 V9 — Linux containers as kernel-requested process isolation, implemented by Docker and Podman

This note argues that a container is not a kernel object: it is an ordinary Linux process that five independent kernel interfaces have been configured to restrict — namespaces (what the process can see), cgroups (what it can consume), capabilities (how much of root it retains), seccomp (which syscalls it may issue), and a union filesystem (what it sees as its root). Docker and Podman are the programs that issue these requests in a re-

peatable order and package the filesystem the process wakes up in. The note traces the lineage back through `chroot`, BSD jails, Solaris Zones and LXC to argue that isolation itself long predated Docker, and that Docker's real contribution was solving distribution instead — an immutable, layered, content-addressed image format plus a registry protocol for moving it between machines. Each of the five mechanisms is backed by the relevant Linux kernel manual page and the OCI image and runtime specifications. The note then contrasts Docker's `client-daemon-containerd-runc` architecture with Podman's `daemonless fork-exec` model, concluding that the difference between the two engines is architectural — who owns the container process — rather than a difference in which kernel primitives either one uses. Its security claims rest on named CVEs (runc container-escape bugs, Dirty Pipe), a peer-reviewed study of 59 container-runtime CVEs finding user namespaces prevented 7 of 9 tested escapes, and vendor and maintainer documentation for the rootless and networking details of each engine.

The note is explicit about what it does not claim. It does not claim containers match virtual-machine isolation: sharing a kernel means the attack surface is the entire syscall interface, and it treats the position that privileged containers cannot be secured as settled rather than as one side of an open debate. Several statements are flagged inline as the author's own inference rather than sourced fact, and a handful of figures — macOS emulation slowdown, some vendor vulnerability-scan percentages — are marked `reported-but-uncorroborated` or explicitly `unverified`. A dedicated "Grounding gap" section names claims the author encountered repeatedly during research and deliberately left out of the note itself for lacking a primary source, among them a specific comparative count of default Linux capabilities between Docker and Podman, which the note says would need direct inspection of each project's source or a runtime `capsh` check before it could be used in teaching material.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**

- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\)](#) — Podman manual page — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman](#) — project README — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun](#) — project README — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**

- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — GigaScience (2021), stable DOI alternative to the publisher URL above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**

- [CNCF org move, what should the final name be? — Podman discussion #28699](#) — Podman core maintainers (2026) **HIGH**

2.10 V10 — Containers implemented via Docker and Podman

The note argues that a container is not a kernel object but an ordinary process configured through five independent Linux mechanisms — namespaces, cgroups, capabilities, seccomp, and union filesystems — and that Docker's real contribution was not isolation, which was already available via `chroot`, jails, Zones and LXC, but a layered, content-addressed image format plus a distribution protocol. It traces the technical history with primary-source dates: `chroot` to 1982, FreeBSD jails to 1999–2000, Solaris Zones to 2004–2005, the Linux mount namespace to 2002, cgroups to 2008, and Docker's 2013 open-sourcing out of the failing dotCloud PaaS. Podman is presented as an architectural critique of Docker rather than a kernel-usage one: the same OCI image format and registry protocol, but no root-owned daemon, implemented instead as `fork-exec` plus a `common` monitor process. Evidence throughout is drawn from primary sources — kernel man pages, the OCI specifications, and first-person accounts from the engineers who built Docker, Podman and `crun` — backed by a security case study of documented `runc` escapes (CVE-2019-5736, CVE-2024-21626, the 2025 `maskedPaths` advisories) and an academic study finding user namespaces prevented 7 of 9 tested runtime-escape exploits. The note records that 102 of 113 factual claims were verified at creation, and it has since had two dated correction passes (2026-09-24 and 2026-09-29) that updated the rootless-storage default, a moved source repository, and a dead publisher link.

The note is explicit about what it does not claim. It does not assert one clean historical lineage from `chroot` through jails and Zones to Linux namespaces — it states that the cgroups and namespaces threads are "three parallel responses to the same shared-hosting pressure" rather than a single inherited design, and it flags disputed dates, such as `chroot`'s 1979-versus-1982 origin, as genuine disagreements rather than resolving them. Several numeric claims are explicitly bounded: `crun`'s performance figures are marked as the project's own self-reported benchmark, vendor vulnerability-scan figures in the 64–90% range are named as unverified and commercially motivated, and a widely repeated claim comparing Podman's and Docker's rootless default capability counts is deliberately excluded as untraceable to any primary source. The note also leaves open exactly which network backend a given Podman release defaults to without checking that release's `containers.conf`, and does not pin exact per-namespace kernel-introduction versions beyond what secondary sources report, since the current man page no longer carries a version column.

Referenced sources

- [namespaces\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#), Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#), Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#), Docker Inc. **HIGH**
- [Docker overview](#), Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#), Docker Inc. **HIGH**
- [Docker build cache](#), Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#), The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#), The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#), Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#), Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#), Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#), Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#), Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#), Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#), Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#), Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#), Poul-Henning Kamp **HIGH**

- [Namespaces](#), Justin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#), Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#), Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#), opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#), Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#), Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#), Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#), Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#), Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#), GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025 \(stable DOI alternate\)](#), GigaScience (2021) **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#), arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#), WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#), Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#), Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#), Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#), The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#), Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#), Microsoft Learn **HIGH**
- [Install Docker Engine](#), Docker Inc. **HIGH**
- [Podman Installation Instructions](#), The Podman project **HIGH**
- [podman-machine\(1\)](#), The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#), Ivan Velichko **MEDIUM**

- [multiarch/qemu-user-static](#), The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11, Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#), Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#), Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#), Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699, Podman core maintainers (2026) **HIGH**

2.11 V11 — Linux Containers: Kernel Mechanisms and the Docker/Podman Architecture

This note argues that a container is not a kernel object but an ordinary Linux process subjected to five independent, separately-documented kernel mechanisms: namespaces (what a process can see), cgroups (what it can consume), capabilities (how much of root it retains), seccomp (which syscalls it may issue), and a union filesystem (what it sees as its root). It grounds these mechanisms in primary kernel documentation and in the OCI image and runtime specifications, and it distinguishes Docker's client/daemon/containerd/runc chain from Podman's daemonless fork-exec model built on libpod, common and crun, arguing that Podman's contribution is architectural rather than a difference in kernel usage. It backs its history with primary sources — chroot's 1982 BSD origin, cgroups' 2006 Google beginnings, Docker's 2013 open-sourcing, Podman's 2026 move out of the `containers` GitHub organisation — and supports its security claims with named CVEs (CVE-2019-5736, the 2024 "Leaky Vessels" bug, the November 2025 runc advisory trio, Dirty Pipe) plus a peer-reviewed study of 59 runtime-escape CVEs, from which it draws the conclusion that user namespaces are the single highest-leverage mitigation, preventing 7 of 9 tested escapes. It also updates course-relevant, time-sensitive facts: rootless Podman's storage now defaults to the kernel's own overlay driver rather than the FUSE fallback its cited 2020 source describes, and its default rootless networking has moved from slirp4netns to pasta.

The note is explicit about what it does not claim. It does not assert that containers match virtual-machine isolation — it states that the isolation boundary is the entire kernel syscall surface, and that privileged containers cannot be made secure by configuration. It flags reasoning it could not fully verify with inline tags: benchmark figures from crun's own repository are marked self-reported, macOS slowdown multipliers and emulated-build-time figures are marked as single-author measurements rather than controlled studies, and vendor vulnerability-scanning figures above 64% are presented as uncorroborated. A dedicated "Grounding gap" section lists what it deliberately excluded for lack of a primary source — including a widely repeated but unsourced comparison of Docker's and Podman's default capability counts — and notes that Podman's own documentation states its rootless-storage kernel threshold inconsistently (5.12 vs. 5.13) rather than resolving it to one figure. Two points are left as genuinely unsettled between competing accounts: whether `chroot` dates to 1979 or 1982, and whether Linux namespaces descend causally from BSD jails or arose in parallel from Plan 9.

Referenced sources

- [namespaces\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#), Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#), Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#), Docker Inc. **HIGH**
- [Docker overview](#), Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#), Docker Inc. **HIGH**
- [Docker build cache](#), Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#), The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**

- [containers/crun — project README](#), The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#), Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#), Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#), Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#), Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#), Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#), Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#), Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#), Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#), Poul-Henning Kamp **HIGH**
- [Namespaces](#), Justin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#), Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#), Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#), opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#), Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#), Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#), Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#), Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#), Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#), GigaScience (2021) **HIGH**
- [DOI: 10.1093/gigascience/giab025](#), stable alternative locator for the above GigaScience article **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#), arXiv preprint **MEDIUM**

- [Moving ARM64 Docker Builds Off QEMU, WarpBuild](#) **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#), Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#), Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#), Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#), The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#), Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#), Microsoft Learn **HIGH**
- [Install Docker Engine](#), Docker Inc. **HIGH**
- [Podman Installation Instructions](#), The Podman project **HIGH**
- [podman-machine\(1\)](#), The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#), Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#), The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11 — overlayfs updates for Linux 5.11](#), Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#), Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#), The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [podman/docs/tutorials/performance.md](#), The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [WSL2-Linux-Kernel releases](#), Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#), Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCf org move, what should the final name be? — Podman discussion #28699](#), Podman core maintainers (2026) **HIGH**

2.12 V12 — Container isolation mechanics and the Docker–Podman architecture split

This note treats a container not as a kernel object but as an ordinary process subjected to five independently configurable kernel mechanisms: namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it retains), seccomp (which syscalls it may issue), and a union filesystem (what it sees as its root). It draws

this definition, and the eight Linux namespace types, directly from the kernel's own `namespaces(7)`, `cgroups(7)`, `capabilities(7)` and `seccomp(2)` manual pages, treating the removal of any one mechanism as leaving "less of a container" rather than something categorically different. On top of that shared kernel substrate, the note argues Docker and Podman diverge architecturally rather than technically: Docker routes every container through a root-owned, long-lived daemon (`dockerd` → `containerd` → `runc`) reached over a REST API, while Podman is daemonless — each invocation forks, execs the OCI runtime (`crun` by default) directly, and hands off to a small `conmon` monitor process, with the container ending up a child of the user's own session rather than of a background service. Both engines use the same OCI image and runtime specifications, which is what the note says makes Podman a drop-in rather than a clone. It backs this with primary-source quotations from Podman's own manual page and each engine's vendor documentation for rootless mode, and weighs the security payoff of user-namespace-based rootless operation against a peer-reviewed study finding it prevented 7 of 9 tested container-escape exploits.

The note does not claim containers match virtual-machine isolation: it states that the isolation boundary is the entire Linux syscall interface, "weaker than the separation of a hypervisor," and catalogues several `runc` CVEs plus one kernel bug (Dirty Pipe) that bypassed the container runtime entirely as evidence. Several figures are explicitly hedged rather than asserted as settled fact: the `crun`-versus-`runc` performance comparison is flagged as the `crun` project's own self-reported, self-selected benchmark; a claim that vendor security scanners find 64–90% of high-download images carrying vulnerabilities is marked unverified because it originates from vendors with a commercial interest in the number. A recurring comparison claim — that Podman's rootless containers get 11 kernel capabilities against Docker's 14 — is deliberately excluded from the note as untraceable to any primary source, with the note instead naming the direct experiment (reading the two projects' default-capability code, or running `capsh --print`) that would settle it. Two further points are logged as unsettled: the exact kernel version each namespace type was introduced in, since the current manual page dropped its version column; and, as of the note's most recent correction pass, which userspace network stack Podman defaults to for rootless containers.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — **HIGH**

- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project — **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative — **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative — **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative — **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. — **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. — **HIGH**
- [Docker overview](#) — Docker Inc. — **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. — **HIGH**
- [Docker build cache](#) — Docker Inc. — **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) — **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation — **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) — **HIGH**
- [What is Podman?](#) — Red Hat (2026) — **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) — **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) — **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) — **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) — **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) — **MEDIUM**
- [History of Containers](#) — Antranig Vartanian — **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp — **HIGH**
- [Namespaces](#) — Iustin Pop (2019) — **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) — **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) — **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers — **HIGH**

- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) — **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) — **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security — **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) — **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) — **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) — **HIGH**
- [DOI: 10.1093/gigascience/giab025](#) — stable alternative locator for the GigaScience article above — **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint — **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild — **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) — **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker — **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) — **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) — **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project — **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn — **HIGH**
- [Install Docker Engine](#) — Docker Inc. — **HIGH**
- [Podman Installation Instructions](#) — The Podman project — **HIGH**
- [podman-machine\(1\)](#) — The Podman project — **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko — **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project — **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) — **HIGH**

- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) — **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024-2026) — **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) — **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) — **HIGH**

2.13 V13 — Containers implemented with Docker and Podman

This note's central claim is that a container is not a kernel object but an ordinary process to which several independent kernel interfaces — namespaces, cgroups, capabilities, seccomp, and a union-mounted root filesystem — have been applied through a repeatable sequence; Docker and Podman are the programs that issue that sequence and package the filesystem the process wakes up in. Each of the five mechanisms is documented separately: namespaces isolate what the process can see (with the user namespace singled out as the load-bearing one for rootless operation, since it is the only type an unprivileged process can create on its own), cgroups bound what it can consume, capabilities restrict how much of root it retains, seccomp filters which syscalls it may issue, and OverlayFS supplies the layered, copy-on-write root filesystem. The note argues the historically interesting part of Docker was not isolation, which namespaces and cgroups already provided by 2008, but distribution — the immutable, content-addressed image format and registry protocol that made deployment deterministic. It traces this lineage from `chroot` through BSD jails, Solaris Zones, and LXC to Docker's 2013 open-sourcing out of the failing dotCloud PaaS, then contrasts Docker's client/daemon/containerd/runc architecture with Podman's daemonless fork-exec model built around `common` and `crun`, backed throughout by kernel man pages, the OCI specifications, vendor documentation, and primary project sources.

The note is equally explicit about what it does not claim. It states plainly that shared-kernel isolation is weaker than a hypervisor boundary, citing multiple documented runc CVEs and kernel-level escapes (e.g. Dirty Pipe) as evidence that no container-runtime fix can close every gap, and that privileged containers should be treated as unsecurable rather than merely risky. It flags several points as unsettled or inferential rather than established: the precise causal relationship between BSD jails and Linux

namespaces, the crun-vs-runc benchmark figures (self-reported by the crun project), and vendor vulnerability-scanning statistics in the 64–90% range, which it reports without independent corroboration. It also records an open "Grounding gap" — a widely repeated but unsourced comparison of Podman's and Docker's default capability counts — that it deliberately excluded for lack of a primary citation, and it notes that exact per-namespace kernel-introduction versions rest on secondary sources because the current `namespaces(7)` man page no longer carries a version column.

Referenced sources

- [namespaces\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#), Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#), Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#), Docker Inc. **HIGH**
- [Docker overview](#), Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#), Docker Inc. **HIGH**
- [Docker build cache](#), Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#), The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#), The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#), Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#), Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#), Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#), Jérôme Petazzoni (2017) **HIGH**

- [Docker grows to harbor innovation](#), Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#), Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#), Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#), Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#), Poul-Henning Kamp **HIGH**
- [Namespaces](#), Justin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#), Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#), Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#), opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#), Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#), Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels – runc \(CVE-2024-21626\)](#), Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#), Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#), Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#), GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#), GigaScience (2021) – stable alternative locator for the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#), arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#), WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#), Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#), Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#), Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima – Linux virtual machines](#), The Lima project (CNCf sandbox) **HIGH**

- [Colima — container runtimes on macOS and Linux with minimal setup](#), Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#), Microsoft Learn **HIGH**
- [Install Docker Engine](#), Docker Inc. **HIGH**
- [Podman Installation Instructions](#), The Podman project **HIGH**
- [podman-machine\(1\)](#), The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#), Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#), The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11 — overlayfs updates for Linux 5.11](#), Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#), Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#), Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#), Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be? — Podman discussion #28699](#), Podman core maintainers (2026) **HIGH**

2.14 V14 — Container Isolation Under Docker and Podman

This note argues that a container is not a distinct kernel object but an ordinary Linux process assembled from five independent, separately maintained kernel mechanisms: namespaces (what a process can see), cgroups (what it can consume), capabilities (how much of root it retains), seccomp (which syscalls it may issue), and a union filesystem composing read-only image layers with one writable layer. It traces the lineage from `chroot` (1982) through BSD jails, Solaris Zones and LXC to Docker, arguing — against the usual clean-chain narrative — that these are parallel responses to shared-hosting pressure rather than one inherited design, and that Docker's real contribution was pairing existing kernel isolation with a layered, content-addressed image format and registry protocol that made deployment deterministic rather than procedural. It then contrasts Docker's client/daemon/containerd/runc architecture with Podman's daemonless fork-exec model built around libpod, common and crun, drawing on kernel man pages, the

OCI image and runtime specifications, vendor documentation from Docker and Red Hat, and first-person accounts from engineers at both projects. Numbered claims also cite CVE advisories (CVE-2019-5736, the 2024 "Leaky Vessels" runc bug, and three 2025 runc CVEs) and a peer-reviewed study of 59 runtime-escape CVEs to support the note's central security claim: user namespaces, i.e. rootless operation, are the single highest-leverage mitigation, having prevented 7 of 9 tested escape exploits in that study, while privileged containers are described by both LWN and a runc maintainer as essentially unsecurable.

The note is explicit about what it does not establish. It flags several passages as [inference] or [unverified], including its own reasoning about why Dockerfile layer ordering matters and vendor-reported vulnerability-percentage figures it declines to corroborate independently. A dedicated "Grounding gap" section lists claims deliberately excluded for lack of primary sourcing — notably a widely repeated but uncited figure comparing the exact kernel-capability counts Podman and Docker grant rootless containers — and open questions it has not settled, such as the precise kernel version at which each namespace type was introduced. It documents its own revision history rather than presenting a single fixed account: an earlier version's description of namespaces as "files" was corrected after author review, and later passes updated claims about rootless-overlay storage support and a project's GitHub organization move once those underlying facts changed, explicitly dating the superseded claims instead of silently deleting them. The note does not take a position on whether Docker or Podman is preferable in the abstract; it states that both rest on identical kernel mechanisms and the same OCI standards, and frames the choice as being about who owns the container process rather than about which engine is more capable.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**

- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\)](#) — Podman manual page — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman](#) — project README — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun](#) — project README — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels](#) — runc (CVE-2024-21626) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**

- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [DOI: 10.1093/gigascience/giab025](#) — stable alternative locator for the GigaScience article above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**

- [CNCF org move, what should the final name be? — Podman discussion #28699](#) — Podman core maintainers (2026) **HIGH**

2.15 V15 — Linux Containers as Kernel Mechanism, Not a Kernel Object

This note's central claim is that a container is not a first-class kernel construct — there is no `struct container` — but an ordinary process that five independent, separately-documented kernel interfaces have been configured to restrict: namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it keeps), `seccomp` (which syscalls it may issue), and a union filesystem (what it sees as `/`). It backs this with the kernel's own definitions of each mechanism from the man pages and the OCI specifications, and traces a history in which `chroot` (1982), FreeBSD jails (2000), Solaris Zones (2004–05) and Linux namespaces (2002–2008) are parallel responses to shared-hosting pressure rather than one inherited lineage, contrary to the commonly told chain. Docker's actual contribution, the note argues, was distribution rather than isolation — a layered, content-addressed image format plus a registry protocol, built by a failing PaaS company (dotCloud) that later open-sourced its container engine. Podman is presented as a critique of Docker's daemon architecture rather than of its kernel usage: it keeps the same OCI image format and runtime contract but replaces the root-owned background daemon with a fork-exec model (`common` plus `crun`), making rootless operation the default rather than a retrofit. The note supports its security claims with named CVEs (2019-5736, 2024-21626, the 2025 runc trio, Dirty Pipe) and a peer-reviewed study finding user namespaces prevented 7 of 9 tested container-escape exploits.

The note is explicit about what it does not claim. It does not treat containers as a hypervisor-grade isolation boundary — it quotes a maintainer stating that privileged containers "are simply never going to be truly secure" — and it flags several figures as unsettled: vendor-reported vulnerability rates of 64–90% for popular images are marked unverified and attributed to sources with a commercial interest in the number; Podman's own documentation gives two different kernel-version thresholds (5.12 vs. 5.13) for native rootless overlay support, and the note records the disagreement rather than resolving it. A "Grounding gap" section lists claims the author encountered repeatedly in secondary sources but deliberately excluded for lack of primary citation, such as a specific capability-count comparison between Podman and Docker. The note also carries a dated log of corrections — to Podman's rootless storage behavior, an organizational URL move, and a networking-backend default — treating itself as a living document rather than a fixed snapshot.

Referenced sources

- [namespaces\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#), Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#), Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#), Docker Inc. **HIGH**
- [Docker overview](#), Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#), Docker Inc. **HIGH**
- [Docker build cache](#), Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#), The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#), The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#), Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#), Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#), Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#), Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#), Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#), Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#), Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#), Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#), Poul-Henning Kamp **HIGH**

- [Namespaces](#), Justin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#), Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#), Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#), opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#), Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#), Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#), Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#), Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#), Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#), GigaScience (2021) **HIGH**
- [DOI: 10.1093/gigascience/giab025](#) — stable alternative locator for the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#), arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#), WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#), Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#), Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#), Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#), The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#), Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#), Microsoft Learn **HIGH**
- [Install Docker Engine](#), Docker Inc. **HIGH**
- [Podman Installation Instructions](#), The Podman project **HIGH**
- [podman-machine\(1\)](#), The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#), Ivan Velichko **MEDIUM**

- [multiarch/qemu-user-static](#), The multiarch project **MEDIUM**
- [Merge tag ov1-update-5.11](#) — overlayfs updates for Linux 5.11, Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#), Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#), Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#), Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699, Podman core maintainers (2026) **HIGH**

2.16 V16 — Containers as a bundle of independent kernel mechanisms

This note's central claim is that a container is not a kernel object but an ordinary process that Docker or Podman has configured, through five independent and separately-documented kernel interfaces, to see and consume less than the machine actually offers: namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it keeps), seccomp (which syscalls it may issue), and a union filesystem (what it sees as `/`). The note backs each mechanism with the relevant Linux manual page or OCI specification text rather than paraphrase — the kernel's own definition of a namespace, the `cgroups(7)` v1/v2 hierarchy distinction, the `capabilities(7)` five-set model, and the `seccomp(2)` strict-versus-filter contrast — and traces the historical lineage (chroot, BSD jails, Solaris Zones, LXC, dotCloud, Docker, Podman) back to primary accounts from the people who built each piece, including Poul-Henning Kamp on jails and Solomon Hykes on Docker's origin as a failing PaaS company's internal tooling. It extends this into a comparative account of Docker's daemon-based architecture against Podman's daemonless fork-exec model, and into a security assessment: shared-kernel isolation is bounded by the entire syscall surface, illustrated with named CVEs (runc container escapes, Dirty Pipe) and a peer-reviewed study finding user namespaces prevented 7 of 9 tested escape exploits.

The note is explicit about what remains unsettled or outside its scope. Several claims are flagged `[inference]` or `[unverified]` rather than presented as fact — for example, vendor-reported vulnerability-percentage figures in the 64–90% range are noted as coming from parties with a commercial interest and are not independently corrobor-

ated. A "Grounding gap" section lists claims the author deliberately excluded for lacking primary sourcing, such as a specific capability-count comparison between Docker and Podman defaults that traced back only to uncited SEO blogs, and leaves open exactly which rootless network backend ships by default in the Podman release students will install. The note does not claim VM-level isolation guarantees for containers, does not treat any single source in its history section as sufficient on its own where two authoritative accounts disagree (e.g., competing kernel-version thresholds for rootless overlay support), and treats its own two Red Hat and project-benchmark sources as authoritative on their own implementations but not neutral on comparative claims against competitors.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**

- [What is Podman? — Red Hat \(2026\)](#) **MEDIUM**
- [Why strace doesn't work in Docker — Julia Evans \(2020\)](#) **HIGH**
- [From dotCloud to Docker — Jérôme Petazzoni \(2017\)](#) **HIGH**
- [Docker grows to harbor innovation — Alex Handy / SD Times \(2014\)](#) **MEDIUM**
- [A Pivot that Worked: The Docker Story — Dan Scholnick / Trinity Ventures \(2019\)](#) **MEDIUM**
- [What Docker adds to the Linux kernel primitives — Stefano Noferi \(2013\)](#) **MEDIUM**
- [History of Containers — Antranig Vartanian](#) **MEDIUM**
- [Jails - High value but shitty Virtualization — Poul-Henning Kamp](#) **HIGH**
- [Namespaces — Iustin Pop \(2019\)](#) **MEDIUM**
- [A container-confinement breakout — Jake Edge / LWN.net \(2019\)](#) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape! — Amazon Web Services \(2019\)](#) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\) — opencontainers/runc maintainers](#) **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881 — Sysdig \(2025\)](#) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview — Cloud Native Computing Foundation \(2025\)](#) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\) — Red Hat Product Security](#) **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes — Michael Reeves, Dave \(Jing\) Tian, Antonio Bianchi, Z. Berkay Celik \(2021\)](#) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs \(2022\)](#) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis — GigaScience \(2021\)](#) **HIGH**
- [10.1093/gigascience/giab025 — GigaScience \(2021\), stable DOI alternate to the publisher URL above](#) **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs — arXiv preprint](#) **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU — WarpBuild](#) **MEDIUM**
- [Embracing the Future with Multi-Arch Containers — Syl Taylor \(2023\)](#) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\) — Docker for Mac issue tracker](#) **MEDIUM**

- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.17 V17 — Containers with Docker and Podman

This note's central claim is that a container is not a kernel object but an ordinary Linux process that five independent, separately configurable kernel mechanisms have been asked to restrict: namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it keeps), seccomp (which syscalls it may issue), and a union-mounted root filesystem (what it sees as `/`). Each mechanism is backed by its own kernel manual page and, for the image and runtime formats, the OCI specifications. The note argues that Docker's real contribution was distribution rather than isolation — namespaces and cgroups already existed by 2008 — and traces the container lineage (`chroot`, BSD jails, Solaris Zones, LXC, Docker) to primary sources including kernel commits, manual pages, and first-person accounts from the engineers who built jails

and Docker itself, correcting a commonly told but causally loose version of that history. It contrasts Docker's daemon-based client-server architecture (`dockerd` → `containerd` → `runc`) with Podman's daemonless fork-exec model (`common` plus `crun`), and shows both engines' rootless modes resting on the same mechanism: the user namespace, the only namespace type that remaps UIDs and the one that lets an unprivileged user obtain the capability needed to create the rest.

The note is explicit about what it does not claim. It does not treat containers as more secure than virtual machines — sharing a kernel means the isolation boundary is only as strong as namespaces and cgroups, with an attack surface spanning the entire syscall interface, a point it backs with named CVEs (runc escapes, Dirty Pipe) rather than asserting in the abstract. Several questions are flagged as unsettled or open rather than resolved: an earlier version of the note misstated Podman's default rootless network backend, which a 2026-09-29 correction now documents as `pasta` replacing `slirp4netns`; a widely repeated claim that Podman's rootless containers get a different capability count than Docker's is deliberately excluded because every version traced back to unsourced secondary blogs; and vendor-reported figures of 64-90% of images carrying vulnerabilities are marked unverified, since they come from security vendors with a commercial interest in the number. The note also declines to call Docker or Podman on macOS or Windows "native," stating plainly that no native container exists off Linux, and marks its emulation and I/O slowdown figures as directional, self-reported measurements rather than controlled benchmarks.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**

- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\)](#) — Podman manual page — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman](#) — project README — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun](#) — project README — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**

- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — GigaScience (2021), stable alternative locator for the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.18 V18 — Linux Containers as Implemented by Docker and Podman

The note's central claim is that a container is not a kernel object at all, but an ordinary process that has been placed under five independent, separately-documented kernel mechanisms — namespaces, cgroups, capabilities, seccomp filtering, and a union-mounted root filesystem — and that Docker and Podman are the programs that issue those requests in a repeatable order and then package the resulting filesystem. Each mechanism is backed by its own primary source: the Linux man pages for namespaces, cgroups, capabilities and seccomp, and the OCI image and runtime specifications for the image/bundle format. The note traces the historical lineage (`chroot` , BSD jails, Solaris Zones, Linux namespaces, cgroups) as largely parallel responses to shared-hosting pressure rather than a single inherited design, and separately documents Docker's origin inside the failing dotCloud PaaS and Podman's daemonless, rootless-first architecture as a deliberate rejection of Docker's root-owned daemon model. It also catalogues concrete evidence for the isolation boundary's limits — named CVEs (runc escapes, Dirty Pipe), a peer-reviewed study of 59 runtime-escape CVEs across 11 runtimes, and measured costs of rootless networking and macOS/Windows virtualization.

The note is explicit about where its own evidence runs out. A recurring comparison claim — that Podman's rootless containers get a specific, smaller number of default capabilities than Docker's — is deliberately excluded because every instance traced back to uncited secondary blogs; the note names the primary sources (`containers.conf` , `moby/oci/defaults.go`) that would need to be read directly to support it. Podman's current default rootless network backend is flagged as unpinned, resting on a 2020 walkthrough and a project README rather than a version-specific release note. Per-namespace kernel introduction versions are sourced to secondary talk notes because the current man page no longer carries a version column. Throughout, the note tags individual claims as `[inference]` (reasoned but not directly sourced), `[reported]` (a source's own figures, not independently measured), or `[unverified]` (vendor figures without independent corroboration) rather than presenting all claims with uniform confidence.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**

- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**

- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [DOI: 10.1093/gigascience/giab025](#) — stable alternative locator for the GigaScience article above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCf) **HIGH**

- podman/docs/tutorials/performance.md — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.19 V19 — Copy-on-write (CoW)

This note argues that copy-on-write is not three separate mechanisms taught in three different course units, but one three-step protocol — share, protect, trap-and-copy — instantiated over different units of sharing: pages in `fork()`, file-backed pages in shared libraries, and blocks, extents, or rows in filesystem or database snapshots. In every case the cost model is the same: work is proportional to units actually written, not units shared, and a fast path skips the copy entirely once a reference count drops to one. The note backs this with primary implementation material (Duke and KU Leuven operating-systems lab exercises describing the page-fault handler, the `PTE_COW` bit, and per-frame reference counting), canonical reference documentation (Linux's `/proc` and `mmap(2)` manuals, Microsoft's memory-protection-constants and ReFS block-cloning pages), and a peer-reviewed measurement paper (EuroSys 2021) showing that `fork()`'s residual cost, once CoW removes the data copy, is walking and reference-counting page tables. It treats the "write storm" — where a workload that looks read-only, such as a garbage-collector mark phase, a reference-count increment, or a metadata `chmod`, silently dirties every shared unit it touches — as core content rather than a footnote, using Redis's own CoW telemetry and CPython's Instagram-driven immortal-objects change (PEP 683) as first-party evidence that the storm is a first-order, sometimes language-changing cost.

The note does not claim CoW is universally cheap or that every instance it names is confirmed to the same evidentiary standard. It flags four points as unsettled or resting on inference rather than a cited measurement: the specific fault-count arithmetic for a 512 MiB test program is presented as arithmetic, not a measurement, because transparent huge pages could invalidate it; the claim that Windows has no supported user-mode `fork()` is inferred from the absence of such an API in the documentation surveyed, rather than stated directly by any source; a search for a Windows per-process equivalent of Linux's `/proc/interrupts` TLB-shootdown counter came up empty, which the note treats as weak evidence of absence rather than proof no such counter exists; and the generalization that tracing garbage collectors on other runtimes, such as the JVM or

V8, dirty pages the same way CPython's collector does is stated as an inference from CPython's confirmed mechanism, not verified against a primary source for those runtimes. The note is explicit that these four points should be checked before being repeated as settled fact in a lecture.

Referenced sources

- [Lab COW: Copy-on-write fork](#) — Duke University, CompSci 310 (2025) **HIGH**
- [Exercise: Copy-on-Write Fork](#) — DistriNet, KU Leuven — OS labs (n.d.) **HIGH**
- [The /proc Filesystem](#) — [The Linux Kernel documentation](#) — The Linux Kernel Organization (2026) **HIGH**
- [Mysteries of Windows Memory Management Revealed](#) — Russinovich & Solomon, Microsoft — conference session deck, re-hosted (n.d.) **MEDIUM**
- [What does "Private Data" define in VMMAP?](#) — Stack Overflow (2015) **MEDIUM**
- [Copy-on-Write, Explained Through fork\(\) and Snapshots](#) — Pickuma (2026-06-10) **MEDIUM**
- [Diagnosing latency issues](#) — Redis (2026) **HIGH**
- [Memory Protection Constants \(WinNT.h\)](#) — Microsoft Learn (2026) **HIGH**
- [On-demand-fork: A Microsecond Fork for Memory-Intensive and Latency-Sensitive Applications](#) — Zhao, Gong & Fonseca, EuroSys (2021) **HIGH**
- [Redis fork/COW memory storm: why persistence doubles memory](#) — Netdata (n.d.) **MEDIUM**
- [mmap\(2\)](#) — [Linux manual page](#) — man7.org / Linux man-pages project (2026) **HIGH**
- [Position Independent Code \(PIC\) in shared libraries](#) — Eli Bendersky (2011) **HIGH**
- [How important is it nowadays to ensure that all my DLLs have non-conflicting base addresses?](#) — Raymond Chen, Microsoft — The Old New Thing (2017) **HIGH**
- [A fine detail on how DLLs are relocated as the result of a base address collision, and consequences](#) — Raymond Chen, Microsoft — The Old New Thing (2017) **HIGH**
- [The Basic Concepts of Snapper](#) — SUSE, SUSE Linux Enterprise Server 16.0 documentation (2026) **HIGH**
- [LVM volume snapshots](#) — SUSE, Storage Administration Guide, SLES 15 SP6 (2026) **HIGH**
- [Well-known Databases Use Different Approaches for MVCC](#) — Amit Kapila / EnterpriseDB (n.d.) **MEDIUM**
- [Block cloning on ReFS](#) — Microsoft Learn (2026) **HIGH**
- [Block Cloning](#) — Win32 apps — Microsoft Learn (2026) **HIGH**

- [Volume Shadow Copy Service](#) — MicrosoftDocs/windowsserverdocs repository (2026) **HIGH**
- [Latency Implications of Virtual Memory](#) — Erik Rigtorp (n.d.) **HIGH**
- [Optimizing the TLB Shutdown Algorithm with Page Access Tracking](#) — Nadav Amit, USENIX ATC (2017) **HIGH**
- [Overlay Filesystem](#) — The Linux Kernel documentation — The Linux Kernel Organization (n.d.) **HIGH**
- [InternalDocs/garbage_collector.md](#) — CPython — Python Software Foundation / CPython contributors (2026) **HIGH**
- [Issue 31558: gc.freeze\(\) — an API to mark objects as uncollectable](#) — Python bug tracker / Łukasz Langa et al. (2017) **HIGH**
- [bpo-31558: Add gc.freeze\(\) — pull request discussion](#) — CPython repository (2017) **HIGH**
- [PEP 683 — Immortal Objects, Using a Fixed Refcount](#) — Python Software Foundation (2022) **HIGH**
- [Introducing Immortal Objects for Python](#) — Eddie Elizondo, Engineering at Meta (2023-08-15) **HIGH**
- [Fixing Copy on Writes from reference counting and immortal objects](#) — CPython issue tracker (2020) **HIGH**
- [mmap — Memory-mapped file support](#) — Python Software Foundation (2026) **HIGH**
- [OverlayFS storage driver](#) — Docker Inc. (2026) **HIGH**
- [CCoW: Optimizing Copy-on-Write Considering the Spatial Locality in Workloads](#) — Electronics, MDPI (2022) **MEDIUM**
- [Copy-on-write](#) — Wikipedia (2026) **MEDIUM**

2.20 V20 — Copy-on-write as one mechanism across scales

This note argues that copy-on-write (CoW) is a single three-step protocol — share a physical unit between two or more owners, protect it against writes by clearing a permission bit or consulting a reference count, and copy on the first write that traps — applied at half a dozen different scales rather than a family of unrelated techniques. It traces the same protocol through five instances (process `fork()`, shared-library/DLL text sharing, filesystem snapshots, database MVCC, and container image layers), each differing only in what unit is shared, how it is protected, and who runs the trap handler, while the cost model stays identical across all of them: $O(\text{units written})$, not $O(\text{units shared})$. Evidence is drawn from primary sources for each instance — university OS-lab

materials for the fork fault sequence, kernel and Win32 documentation for `/proc/<pid>/smaps` and `PAGE_WRITECOPY` semantics, vendor filesystem documentation for Btrfs/LVM/ReFS snapshot behaviour, and CPython's own internal documentation and bug tracker for the reference-counting case. The note gives comparable weight to a second claim: CoW silently converts a read-heavy workload into a write-heavy one — the "write storm" — once writes spread across a shared region, illustrated concretely by CPython's refcounting (where a pure iteration writes to every object's header), Redis's `BGSAVE` fork, and Docker's `overlayfs` `copy_up`.

The note is explicit about what it does not establish. It does not re-derive the CoW fault sequence or the minor/major fault classification, deferring both to two companion vault notes it cites rather than restates. Several claims are flagged as inference or unverified rather than sourced: the arithmetic prediction of fault counts for a given page count and write stride, the claim that Windows has no supported user-mode `fork()`, the absence of a per-process Windows counterpart to Linux's `/proc/interrupts` TLB-shootdown row (noted as weak evidence of absence, not confirmed), and the generalisation of CPython's mark-phase write pattern to other tracing garbage collectors such as the JVM or V8, which was not checked against a primary source for those runtimes. It also declines to adjudicate a naming ambiguity it flags rather than resolves: "copy-on-write" is used in the industry for two structurally different data-movement patterns — redirect-on-write and copy-on-first-write — and the note treats conflating them as a common error rather than asserting either is the more correct usage.

Referenced sources

- [Lab COW: Copy-on-write fork](#) — Duke University, CompSci 310 (2025). HIGH
- [Exercise: Copy-on-Write Fork](#) — DistriNet, KU Leuven — OS labs (n.d.). HIGH
- [The /proc Filesystem](#) — The Linux Kernel documentation — The Linux Kernel Organization (2026). HIGH
- [Mysteries of Windows Memory Management Revealed](#) — Russinovich & Solomon, Microsoft — conference session deck, re-hosted (n.d.). MEDIUM
- [What does "Private Data" define in VMMAP?](#) — Stack Overflow (2015). MEDIUM
- [Copy-on-Write, Explained Through fork\(\) and Snapshots](#) — Pickuma (2026-06-10). MEDIUM
- [Diagnosing latency issues](#) — Redis (2026). HIGH
- [Memory Protection Constants \(WinNT.h\)](#) — Microsoft Learn (2026). HIGH
- [On-demand-fork: A Microsecond Fork for Memory-Intensive and Latency-Sensitive Applications](#) — Zhao, Gong & Fonseca, EuroSys (2021). HIGH
- [Redis fork/COW memory storm: why persistence doubles memory](#) — Netdata (n.d.). MEDIUM

- [mmap\(2\) — Linux manual page — man7.org / Linux man-pages project \(2026\).](#) **HIGH**
- [Position Independent Code \(PIC\) in shared libraries — Eli Bendersky \(2011\).](#) **HIGH**
- [How important is it nowadays to ensure that all my DLLs have non-conflicting base addresses? — Raymond Chen, Microsoft — The Old New Thing \(2017\).](#) **HIGH**
- [A fine detail on how DLLs are relocated as the result of a base address collision, and consequences — Raymond Chen, Microsoft — The Old New Thing \(2017\).](#) **HIGH**
- [The Basic Concepts of Snapper — SUSE, SUSE Linux Enterprise Server 16.0 documentation \(2026\).](#) **HIGH**
- [LVM volume snapshots — SUSE, Storage Administration Guide, SLES 15 SP6 \(2026\).](#) **HIGH**
- [Well-known Databases Use Different Approaches for MVCC — Amit Kapila / EnterpriseDB \(n.d.\).](#) **MEDIUM**
- [Block cloning on ReFS — Microsoft Learn \(2026\).](#) **HIGH**
- [Block Cloning — Win32 apps — Microsoft Learn \(2026\).](#) **HIGH**
- [Volume Shadow Copy Service — MicrosoftDocs/windowsserverdocs repository \(2026\).](#) **HIGH**
- [Latency Implications of Virtual Memory — Erik Rigtorp \(n.d.\).](#) **HIGH**
- [Optimizing the TLB Shutdown Algorithm with Page Access Tracking — Nadav Amit, USENIX ATC \(2017\).](#) **HIGH**
- [Overlay Filesystem — The Linux Kernel documentation — The Linux Kernel Organization \(n.d.\).](#) **HIGH**
- [InternalDocs/garbage_collector.md — CPython — Python Software Foundation / CPython contributors \(2026\).](#) **HIGH**
- [Issue 31558: gc.freeze\(\) — an API to mark objects as uncollectable — Python bug tracker / Łukasz Langa et al. \(2017\).](#) **HIGH**
- [bpo-31558: Add gc.freeze\(\) — pull request discussion — CPython repository \(2017\).](#) **HIGH**
- [PEP 683 — Immortal Objects, Using a Fixed Refcount — Python Software Foundation \(2022\).](#) **HIGH**
- [Introducing Immortal Objects for Python — Eddie Elizondo, Engineering at Meta \(2023-08-15\).](#) **HIGH**
- [Fixing Copy on Writes from reference counting and immortal objects — CPython issue tracker \(2020\).](#) **HIGH**
- [mmap — Memory-mapped file support — Python Software Foundation \(2026\).](#) **HIGH**

- [OverlayFS storage driver](#) — Docker Inc. (2026). HIGH
- [CCoW: Optimizing Copy-on-Write Considering the Spatial Locality in Workloads](#) — Electronics, MDPI (2022). MEDIUM
- [Copy-on-write](#) — Wikipedia (2026). MEDIUM

2.21 V21 — The System Call Boundary Between Programs and the Kernel

This note establishes that "the system call boundary" is actually two distinct boundaries that get conflated: a **mechanism boundary**, the hardware fact of where the CPU changes privilege level (on x86-64, the `syscall` instruction transitioning ring 3 to ring 0, documented in Intel's own architecture reference), and a **contract boundary**, the policy choice of where an operating system promises not to change its interface. Its central claim is that Linux and Windows draw these two boundaries in different places, and that this single design decision explains most of the visible differences between the platforms. Linux collapses the two into one place: its numbered system calls are simultaneously the mechanism and the permanent, append-only contract, a guarantee stated directly in the kernel's own documentation and by a lead kernel maintainer, and confirmed empirically by binaries built against decades-old kernels still running today. Windows deliberately separates them: the mechanism sits in `ntdll.dll`, one layer below the documented Win32 contract, and Microsoft's own driver documentation warns developers away from undocumented native entry points whose underlying service numbers are shown, by a long-running empirical dataset tracking every Windows release, to change freely between builds. The note traces this asymmetry down to a structural cause — Linux ships without a bundled C library and so the kernel itself must carry the stable promise, while Windows ships kernel and libraries as one product and is free to place the contract wherever convenient — and shows it determines which tracing tools can even exist on each platform.

The note is explicit about where its claims stop. It does not claim the mechanism-boundary crossing is always expensive: it documents the Linux `vDSO`, which serves certain calls from a shared page with no privilege transition at all, and `io_uring` batching, which can drive steady-state I/O submission cost toward zero, as real exceptions to a naive "syscalls are costly" framing. Specific cycle-count figures for crossing overhead are drawn from single-machine or single-study benchmarks and are flagged in the note itself as illustrative rather than generalizable, with an explicit instruction to measure one's own hardware. The claim that Windows numbers are unstable is treated as settled by primary vendor documentation and empirical tracking, but the note separately flags a genuine, unresolved disagreement among practitioners over how durable `ntdll`'s *exported functions* (as opposed to the numbers beneath them) actually are in

practice, presenting both sides rather than adjudicating. The note also confines its detailed register-level mechanism discussion to x86-64 and flags AArch64/RISC-V as sharing the same shape with different specifics, left untaught here.

Referenced sources

- [Anatomy of a system call, part 1](#) — David Drysdale / LWN.net (2014) **HIGH**
- [Anatomy of a system call, part 2](#) — David Drysdale / LWN.net (2014) **HIGH**
- [SYSCALL — Fast System Call \(x86 instruction reference\)](#) — Felix Cloutier, transcribed from the Intel SDM **HIGH**
- [x86-64 Syscall Entry](#) — Linux Kernel Internals — kernel-internals.org **MEDIUM**
- [The Linux Kernel Driver Interface \(stable-api-nonsense\)](#) — Greg Kroah-Hartman / Linux kernel documentation **HIGH**
- [Add kernel ↔ userspace ABI stability documentation](#) — Greg Kroah-Hartman / LWN.net (2006) **HIGH**
- [x86 Assembly / Interfacing with Linux](#) — Wikibooks contributors **MEDIUM**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange **MEDIUM**
- [Libraries and Headers — Windows drivers](#) — Microsoft Learn **HIGH**
- [winternl.h header](#) — Win32 apps — Microsoft Learn **HIGH**
- [NTDLL Functions](#) — Geoff Chappell **HIGH**
- [Windows X86-64 System Call Table \(XP through Windows 11 and Server\)](#) — Mateusz "j00ru" Jurczyk **HIGH**
- [Microsoft Windows library files — NTDLL.DLL](#) — Wikipedia contributors **MEDIUM**
- [Windows X86-64 System Call Table — discussion thread](#) — Hacker News contributors **LOW**
- [strace — linux syscall tracer](#) — The strace project **HIGH**
- [ptrace and Syscall Interception](#) — Linux Kernel Internals — kernel-internals.org **MEDIUM**
- [Process Monitor — Sysinternals](#) — Microsoft Learn / Sysinternals **HIGH**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr **MEDIUM**
- [What Makes System Calls Expensive](#) — Abhinav Upadhyay (2025) **MEDIUM**
- [vDSO and Virtual System Calls](#) — Linux Kernel Internals — kernel-internals.org **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud engineering **MEDIUM**
- [Why you should use io_uring for network I/O](#) — Red Hat Developers (2023) **HIGH**

- [Speed up open\(\).read\(\) pattern by reducing the number of system calls](#) — cpython#120754 — CPython issue tracker (2024) **HIGH**
- [win32file.CreateFile](#) — PyWin32 documentation — Mark Hammond / Tim Golden **MEDIUM**
- [ltrace: When strace Is Not Enough](#) — Satesh / unixy.io (2026) **MEDIUM**
- [System Calls and the User-to-Kernel Transition](#) — Yunolay (2026) **MEDIUM**
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com (2026) **LOW**
- [In Linux x86_64, are syscalls and int 0x80 related?](#) — Stack Overflow **MEDIUM**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Palo Alto Networks / Unit 42 (2024) **HIGH**
- [Linux kernel system calls table](#) — Marcin Juskiewicz **HIGH**
- [io_uring Interface Overview](#) — EmergentMind, summarizing Jasny et al. **LOW**

2.22 V22 — The Leaky Boundary Between OS Abstractions and the Machine

The note's claim is that every higher-level abstraction over the operating system — a file handle, a thread, a memory-mapped region, a profiler's time attribution — is a contract with an unwritten "scope clause," and that the leak is not a defect but the abstraction revealing what was always underneath it. It builds this case through five concrete failure modes, each backed by a named tool and observable evidence rather than by reasoning alone. A successful `write()` only means the kernel accepted the bytes into its page cache, not that they are durable; only `fsync()` (and, separately, an `fsync()` on the containing directory) forces that, shown via `strace/blktrace` on Linux and Process Monitor/ETW on Windows, and documented in both platforms' own manuals. Closing a file releases the handle without guaranteeing a flush and can silently discard errors from earlier writes. "Thread" names at least three different runtime objects — a GIL-serialized CPython OS thread, a libuv worker drawn from a four-thread pool in Node.js, or a Windows fiber/UMS unit, the latter of which Microsoft has since withdrawn entirely. `mmap` trades syscalls for page faults, so it beats `read()` only for workloads with resident, repeated access, not for a single sequential pass. And CPU profilers misattribute time spent blocked or inside the kernel, which is why counting syscalls first is presented as a cheaper diagnostic than trusting a profiler's percentages. Each case is grounded in Linux/Windows primary documentation plus measured examples — Evan Jones's `blktrace` study, Daniel Lemire's `read/mmap` benchmarks, the Firefox 3 ext3 `fsync` stall, and Facebook Scuba's `mmap`-based restart fix.

The note is explicit about what it does not claim. It treats every specific fact as version-scoped rather than permanent: Windows User-Mode Scheduling existed and was later removed, `io_uring` changed what a "syscall count" even measures, and CPython's GIL is being loosened starting with 3.12/3.13, so it asks readers to re-verify rather than treat any claim as timeless. It also notes that observation itself is not neutral — `strace` and ETW both perturb the very behavior they measure — and that some leaks (device write caches, NUMA, per-CCX cache slicing) sit below the OS entirely, with "learn one layer down" having no fixed floor. One numeric claim in the note (a rough per-syscall cost in cycles) is flagged in-text as unsourced beyond aggregator content and left as an open item to backfill. The note does not argue for discarding abstractions; it presents trusting them, pushing the guarantee into a dependency, bypassing the OS cache, or restructuring the boundary's economics as equally defensible responses depending on context.

Referenced sources

- [close\(2\) — Linux manual page](#), Michael Kerrisk / man7.org (2024) **HIGH**
- [fsync\(2\) — Linux manual page](#), Michael Kerrisk / man7.org (2024) **HIGH**
- [Flushing System-Buffered I/O Data to Disk](#), Microsoft Learn (2021) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#), Microsoft Learn (2023) **HIGH**
- [Durability: Linux File APIs](#), Evan Jones (2020) **MEDIUM**
- [The Secret Life of fsync](#), Andrey Pechkurov / puzpuzpuz.dev (2022) **MEDIUM**
- [Revised notes on the reliability of FlushFileBuffers](#), Raymond Chen / Microsoft DevBlogs (2017) **HIGH**
- [fsync, fdatsync, and O_SYNC](#), kernel-internals.org (2025) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#), Python Software Foundation (2026) **HIGH**
- [Bypassing the GIL for Parallel Processing in Python](#), Real Python (2024) **MEDIUM**
- [About Processes and Threads](#), Microsoft Learn (2023) **HIGH**
- [Fibers](#), Microsoft Learn (2023) **HIGH**
- [User-Mode Scheduling](#), Microsoft Learn (2022) **HIGH**
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#), The Practical Developer (2026) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#), Microsoft Learn (2022) **HIGH**
- [Which is fastest: read, fread, ifstream or mmap?](#), Daniel Lemire (2012) **HIGH**
- [Latency implications of virtual memory](#), Erik Rigtorp (2020) **MEDIUM**
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#), linux-vox.com (2025) **LOW**

- [File Access: Memory-Mapped vs. I/O System Call Performance](#), Baeldung on Linux (2024) **MEDIUM**
- [strace\(1\) — Linux manual page](#), strace project / man7.org (2024) **HIGH**
- [DTrace on Windows](#), Microsoft Learn (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#), Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#), systeminternals.dev (2026) **MEDIUM**
- [The Law of Leaky Abstractions](#), Joel Spolsky (2002) **HIGH**
- [There Is a Mountain Lion in the Datacenter](#), HFT University (2026) **LOW**
- [Sysinternals Suite 2026.19.08](#), Neowin (2026) **MEDIUM**
- [How Python's GIL Works](#), Codefinity (2025) **LOW**

2.23 V23 — Where operating-system abstractions leak

This note argues that every language-level abstraction over the operating system — a file handle, a thread, a memory-mapped region — is a contract with an unwritten scope clause, and that the leak is not a defect but the abstraction telling the programmer what was underneath it all along. It builds this claim through five fixed-shape cases (belief, reality, observation, cost) on Linux and Windows treated as structurally equal: a successful `write()` that is not durable, because the kernel only takes the data into the page cache and `fsync` — plus a directory `fsync` for new files — is what actually forces it to the device; a `close()` that neither flushes nor safely reports earlier write errors, so higher-level constructs like a Python `with` block or a C# `using` statement discard the last error along with the handle; a "thread" that names at least three different runtime objects, so CPython's GIL-serialized OS threads and Node's four-thread libuv pool both violate the naive "N threads means N things running at once" belief in different ways; an `mmap` that is not simply faster than `read()`, because it trades syscalls for page faults and wins only on repeated random access to resident data, not on a single sequential pass; and a CPU profiler that attributes time to the wrong call site because it cannot see time spent blocked in the kernel, making syscall count rather than bytes moved the diagnostic quantity. Each case is grounded in primary sources — Linux man pages, Microsoft Learn documentation, and measured practitioner benchmarks — and paired with a named observation tool (`strace`, `blktrace`, Process Monitor, ETW) so the claim is falsifiable rather than asserted.

The note is explicit about what it does not settle. It does not claim OS abstractions are worthless — trusting the abstraction and covering gaps operationally is called a defensible default that only fails where the failure mode is silent. It does not resolve whether

`mmap` is faster than `read()` in general, presenting the Lemire and linuxvox sources as a genuine cross-source disagreement resolved only by measuring one's own access pattern, and it cites Facebook's Scuba restart optimization as a case where `mmap` wins decisively. It flags one of its own claims — that a mode switch costs on the order of hundreds of cycles — as unsupported by a traceable primary measurement, recommending students verify it themselves rather than treating the figure as established. And it repeatedly notes that the underlying mechanisms move: User-Mode Scheduling was removed in Windows 11, `io_uring` changed what "syscall count" means, and CPython's GIL is being unwound across recent versions, so it treats every specific claim as version-scoped rather than timeless, and cautions that even its own cross-platform comparisons will degrade as the platforms change.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) HIGH
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) HIGH
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021) HIGH
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023) HIGH
- [Durability: Linux File APIs](#) — Evan Jones (2020) MEDIUM
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022) MEDIUM
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017) HIGH
- [fsync, fdatasync, and O_SYNC](#) — kernel-internals.org (2025) MEDIUM
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026) HIGH
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024) MEDIUM
- [About Processes and Threads](#) — Microsoft Learn (2023) HIGH
- [Fibers](#) — Microsoft Learn (2023) HIGH
- [User-Mode Scheduling](#) — Microsoft Learn (2022) HIGH
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026) MEDIUM
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022) HIGH
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012) HIGH
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020) MEDIUM
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linuxvox.com (2025) LOW
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024) MEDIUM

- [strace\(1\) — Linux manual page — strace project / man7.org](#) (2024) **HIGH**
- [DTrace on Windows — Microsoft Learn](#) (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations — Yarden Shafir / Winsider Seminars & Solutions](#) (2021) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven — systeminternals.dev](#) (2026) **MEDIUM**
- [The Law of Leaky Abstractions — Joel Spolsky](#) (2002) **HIGH**
- [There Is a Mountain Lion in the Datacenter — HFT University](#) (2026) **LOW**
- [Sysinternals Suite 2026.19.08 — Neowin](#) (2026) **MEDIUM**
- [How Python's GIL Works — Codefinity](#) (2025) **LOW**

2.24 V24 — Dev Containers

This note establishes dev containers as a reproducible, containerized development environment defined by the open `devcontainer.json` specification, and it separates the specification from its implementations: the spec fixes the schema, the Features/Templates distribution model, and lifecycle-hook semantics, while VS Code, GitHub Codespaces, the JetBrains IDE family, DevPod, and the Dev Container CLI each consume that same specification without locking teams to one vendor. Its central empirical claim — that this reduces "works on my machine" friction — rests on an action design research study at a Finnish company (Ghanbari et al., Journal of Systems and Software, 2026) that measured environment setup time falling from a full workday to roughly fifteen minutes after adopting a containerized approach. The note also tracks the ecosystem's edges as they actually stand rather than as marketed: JetBrains support spans all major IDEs but carries documented gaps; upstream DevPod is dormant (last stable release March 2025, one commit to its default branch since mid-2025), with maintenance load having moved to a community fork and then to a rewritten successor, Devsy; and Apple's `container` tool matured rapidly through 2026 while shipping eleven security advisories across two releases, which the note reads as a maturity signal that cuts both ways — audited and responsive, but with an attack surface still being mapped.

The note is explicit about what it does not claim. It does not assert that dev containers suit every project: for small, single-language repositories it argues version managers like `asdf` or `mise` may deliver sufficient reproducibility at lower overhead, reserving the strongest case for polyglot, multi-service, or large-team settings. It flags industry-wide adoption data as scarce — GitHub's most recent Codespaces-specific usage figures date to 2023, and no independent survey isolates dev-container adoption from general containerization trends. It withholds a governance verdict on the specification, noting the balance between Microsoft's stewardship and community input lacks documenta-

tion comparable to OCI or CNCF. And on DevPod, it stops short of calling the project dead: the "unmaintained" characterization is sourced to community issues and an independent review rather than any statement from its vendor, vCluster Labs, and the note says so rather than asserting a deprecation that has not been issued. Rootless Docker compatibility with the full Features ecosystem is likewise marked untested rather than assumed to work.

Referenced sources

- [Development Container Specification](#) — Dev Container Spec Maintainers HIGH
- [devcontainers/spec](#) — GitHub Repository — Dev Container Spec Maintainers (2022–present) HIGH
- [Development Containers Overview](#) — Dev Container Spec Maintainers HIGH
- [Using development environment as code for enhancing developer experience: An action design research study](#) — Ghanbari, H., Terimaa, T., Koskinen, K. — Journal of Systems and Software, Vol. 236 (2026) HIGH
- [Dev Container Features Reference](#) — Dev Container Spec Maintainers HIGH
- [@devcontainers/cli](#) — npm — Dev Container Spec Maintainers HIGH
- [devcontainers/ci](#) — GitHub Action — Dev Container Spec Maintainers (2021–present) HIGH
- [What is DevPod?](#) — Loft Labs / vCluster Labs HIGH
- [Introducing DevPod](#) — Codespaces but Open Source — Loft Labs MEDIUM
- [Docker Pricing FAQ](#) — Docker, Inc. (2024) HIGH
- [Dev Container Limitations](#) — IntelliJ IDEA Documentation — JetBrains (2025) HIGH
- [Add further JetBrains support](#) — Issue #603 — devcontainers.github.io (2025) MEDIUM
- [GitHub Codespaces](#) — GitHub / Microsoft HIGH
- [GitHub Repository Statistics: Market Data Report 2026](#) — Gitnux (2026) MEDIUM
- [Devenv: Reproducible Development Environments with Nix](#) — DevTools Guide (2026) MEDIUM
- [Hands-On Review: Devcontainers vs Nix vs Distrobox for Web Teams \(2026\)](#) — web-dev.cloud (2025) MEDIUM
- [Add security settings to deny dangerous devcontainer configurations](#) — Issue #287765 — microsoft/vscode (2026) HIGH
- [Docker Security Cheat Sheet](#) — OWASP — OWASP Foundation HIGH
- [Apple gives Mac devs a WSL-ish thing to call their own](#) — The Register (2026-06-11) HIGH
- [Discover container machines](#) — WWDC26 Session 389 — Apple Inc. (2026) HIGH

- [apple/container — Technical Overview and How-To documentation](#) — Apple Inc. (2026) **HIGH**
- [What's New in IntelliJ IDEA 2026.1](#) — JetBrains (2026) **HIGH**
- [Dev Containers — JetBrains Marketplace plugin \(versions and release notes\)](#) — JetBrains (2026) **HIGH**
- [Releases — loft-sh/devpod](#) — Loft Labs / vCluster Labs (2025) **HIGH**
- [2025 Docker State of Application Development Report](#) — Docker, Inc. (2025) **MEDIUM**
- [What's New in IntelliJ IDEA 2026.2](#) — JetBrains (2026-07-16) **HIGH**
- [Release: apple/container 1.1.0](#) — Apple Inc. (2026-07-06) **HIGH**
- [Release: loft-sh/devpod v0.7.0-alpha.34](#) — Loft Labs / vCluster Labs (2025-06-23) **HIGH**
- [Release: apple/container 1.2.0](#) — Apple Inc. (2026-07-29) **HIGH**
- [Release: apple/container 1.3.0](#) — Apple Inc. (2026-08-24) **HIGH**
- [Release: apple/container 1.3.1](#) — Apple Inc. (2026-08-29) **HIGH**
- [apple/container — Container Machine documentation](#) — Apple Inc. (2026) **HIGH**
- [apple/container — Networking documentation](#) — Apple Inc. (2026) **HIGH**
- [\[Request\]: k8s plugin — Issue #2043](#) — apple/container (2026-07-30) **HIGH**
- [Still Maintained? — Issue #1915](#) — loft-sh/devpod, opened by brunocascio (2025-09-11) **HIGH**
- [Community Devpod — Issue #1946](#) — loft-sh/devpod, opened by skevetter (2025-11-30) **HIGH**
- [This Repo is Unmaintained — Issue #1992](#) — loft-sh/devpod, opened by skevetter (2026-07-23) **HIGH**
- [READ THIS FIRST — I AM DEVELOPING DEVSYS — Issue #773](#) — skevetter/devpod, opened by skevetter (2026-07-02) **HIGH**
- [Releases — skevetter/devpod](#) — skevetter (2025-2026) **HIGH**
- [Devsy — Devcontainer Workspaces, Any Backend, No Lock-In](#) — devsy-org / skevetter (2026) **HIGH**
- [We're Now vCluster Labs](#) — vCluster Labs (formerly Loft Labs) (2025-08-15) **HIGH**
- [docs\(platform\): remove devpod documentation from 4.5+ — commit 7f69128](#) — loft-sh/vcluster-docs (2026-01-12) **HIGH**
- [DevPod Review — Client-Side and Provider-Agnostic CDE](#) — InfraGap.com (2026-07-26) **MEDIUM**

2.25 V26 — Processes, threads, and schedulers on Linux and Windows

This note argues that processes, threads, and scheduling are three views of one kernel data structure, and that the differences between Linux and Windows are structural rather than cosmetic. On Linux, `fork()` and `pthread_create()` both resolve to the same `clone()` syscall with a different flag word, so the kernel never distinguishes a "process" from a "thread" — both are `task_struct` tasks. Windows has no `fork()` in its public API; the note corrects the common claim that the NT kernel cannot do this, showing instead that the kernel supports it natively while the Win32 API and the user-mode stack do not, so an efficient fork requires every layer beneath it to be fork-aware. Blocking is treated the same way: not a property of a device or socket, but a scheduler decision to move a task onto a wait queue and later back. The note traces the fair-share scheduler across a version boundary — CFS through kernel 6.5, replaced by EEVDF from 6.6 — and describes EEVDF's eligibility-plus-deadline mechanism, including the 6.12 addition of `sched_ext`, which lets a BPF program supply the scheduling policy itself. It then covers runtime-level concurrency (Go's goroutines and netpoller, Java's virtual threads and carrier-thread pinning, Python's GIL and its opt-in free-threaded build) and the readiness-versus-completion split between `epoll / kqueue` and IOCP/ `io_uring`. Throughout, the note insists on pairing each mechanism with a way to observe it happening — `strace`, DTrace, ETW, eBPF — treating a tracer's blind spots as evidence of where an abstraction is doing real work.

The note does not claim that either operating system's model is superior, and it flags several of its own claims as less than fully grounded. Two points carry explicit uncertainty markers rather than being stated flatly: the 30-second `sched_ext` watchdog default, and the observation that scheduler-tracing now has to account for a policy that can be swapped at runtime. A closing "Grounding gap" section lists claims the author says rest on general systems knowledge rather than a retrieved primary source and should be backfilled before use in teaching material — among them, the exact mechanics of Python's `multiprocessing` spawn behaviour on Windows, platform differences in `asyncio`'s proactor loop, and the signal-based mechanism behind Go's asynchronous goroutine preemption. Elsewhere the note is careful about scope: the commonly cited ~20x overhead for `strace` is presented as an order-of-magnitude estimate rather than a controlled benchmark, and Meta's reported 28% p99 latency improvement from a custom `sched_ext` policy is described as specific to one workload and one hand-written policy, not as evidence that custom schedulers generally outperform EEVDF. Where a widely repeated secondary claim about EEVDF (a `latency_nice` attribute) is checked against the kernel header and found not to exist, the note states that as a falsification rather than treating the matter as settled either way.

Referenced sources

- [clone\(2\) — Linux manual page](#), Michael Kerrisk / Linux man-pages project **HIGH**
- [Launching Linux threads and processes with clone](#), Eli Bendersky (2018) **HIGH**
- [A fork\(\) in the road](#), Andrew Baumann, Jonathan Appavoo, Orran Krieger, Timothy Roscoe — HotOS XVII (2019) **HIGH**
- [Teaching Operating Systems Concepts with Execution Visualization](#), Francis Giraldeau, Michel R. Dagenais, Hanifa Boucheneb — ASEE **HIGH**
- [Scheduler Internals — Deep into Linux and Beyond](#), compiled kernel-internals reference **MEDIUM**
- [The Process API — fork, exec, and wait](#), Operating Systems: The Abstraction Design Approach **MEDIUM**
- [Processes — UNIX and Windows process models](#), reproduced technical-book chapter **MEDIUM**
- [DTrace on Windows](#), Microsoft Learn **HIGH**
- [Monitoring Windows System Calls Using DTrace & XPerf](#), Apriorit **MEDIUM**
- [CFS: Completely fair process scheduling in Linux](#), Marty Kalin / Opensource.com (2019) **MEDIUM**
- [The Windows NT kernel architecture](#), Julio Merino (2024) **MEDIUM**
- [Green thread](#), Wikipedia **MEDIUM**
- [What is the difference between green threads and virtual threads?](#), Stack Overflow **MEDIUM**
- [Go I/O Optimization: goroutine-per-connection, netpoller & the Reader/Writer Interface](#), James Li / DEV Community **MEDIUM**
- [Exploration of Java Virtual Threads and Performance Analysis](#), Alibaba Cloud **MEDIUM**
- [Virtual Threads after JDK 24: What Changed for Production Java](#), InfoQ (2026) **MEDIUM**
- [io_uring vs epoll — Linux Kernel Internals](#), kernel-internals.org **MEDIUM**
- [io_uring for High-Performance DBMSs: When and How to Use It](#), arXiv preprint **MEDIUM**
- [A Deep Dive into Zero-Copy Networking and io_uring](#), Jatin Umamtora / Medium **LOW**
- [actually, BSD kqueue is a mountain of technical debt](#), Ariadne Conill (2021) **MEDIUM**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#), Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**

- [I/O Completion Ports](#), Microsoft Learn **HIGH**
- [CreateIoRing / ioringapi](#), Microsoft Learn **HIGH**
- [IORING_OP_CODE enumeration \(ntioring_x.h\)](#), Microsoft Learn **HIGH**
- [The Depths of Windows I/O: IOCP and the .NET Thread Pool](#), ComComponent **MEDIUM**
- [strace\(1\) — Linux manual page](#), strace project / Linux man-pages **HIGH**
- [Linux scheduling and using strace on Linux](#), Frits Hoogland **MEDIUM**
- [strace — Tracing Linux Syscalls with ptrace, and When to Use bpftrace Instead](#), Systems Explained (2026) **MEDIUM**
- [Teaching Operating Systems with FreeBSD Through Tracing, Analysis, & Experimentation](#), Robert N. M. Watson, George Neville-Neil — FreeBSD Journal / USENIX ;login: (2016–17) **HIGH**
- [Go execution tracer](#), Gopher Academy (2017) **MEDIUM**
- [io_uring Rootkit Bypasses Linux Security Tools](#), ARMO (2025) **MEDIUM**
- [io_uring: Linux Performance Boost or Security Headache?](#), Upwind (2025) **MEDIUM**
- [EEVDF Scheduler — The Linux Kernel documentation](#), Linux kernel documentation **HIGH**
- [Linux 6.6 changelog](#), Kernelnewbies **MEDIUM**
- [Completing the EEVDF scheduler](#), Jonathan Corbet / LWN.net (2024) **HIGH**
- [PATCH sched/Documentation: Modify the knob to base_slice_ns](#), Shrikanth Hegde, linux-kernel mailing list (2023) **HIGH**
- [linux/kernel/sched/syscalls.c \(v7.2\)](#), Linux kernel source **HIGH**
- [linux/include/uapi/linux/sched.h \(v7.2\)](#), Linux kernel source **HIGH**
- [linux/kernel/sched/features.h \(v7.2\)](#), Linux kernel source **HIGH**
- [Extensible Scheduler Class — The Linux Kernel documentation](#), Linux kernel documentation **HIGH**
- [sched_ext: a BPF-extensible scheduler class \(Part 1\)](#), Changwoo Min / Igalia **MEDIUM**
- [Modernizing the Meta Ads Service With an Open-Source Kernel Scheduler](#), Meta Engineering (2026) **HIGH**
- [dtrace — Windows commands](#), Microsoft Learn **HIGH**
- [What's new in Windows Server 2025](#), Microsoft Learn **HIGH**
- [IORING_VERSION enumeration](#), Microsoft Learn **HIGH**
- [IsIoRingOpSupported function](#), Microsoft Learn **HIGH**
- [linux/include/uapi/linux/io_uring.h](#), Linux kernel source (counted at tags v5.6, v6.12 and v7.2) **HIGH**

- [Documentation for /proc/sys/kernel/](#), Linux kernel documentation **HIGH**
- [Technology Preview features — RHEL 10.0 Release Notes](#), Red Hat **HIGH**
- [Technology previews — RHEL 9.4 Release Notes](#), Red Hat **HIGH**
- [Exploit released for new PinTheft Arch Linux root escalation flaw](#), BleepingComputer (20 May 2026) **MEDIUM**
- [Python support for free threading](#), CPython documentation **HIGH**
- [PEP 779 — Criteria for supported status for free-threaded Python](#), Python Enhancement Proposal, status Final, Python 3.14 **HIGH**
- [Linux 6.12 changelog](#), Kernelnewbies **MEDIUM**

2.26 V27 — Linux containers, and the Docker/Podman architectures built on them

The note's central claim is that a container is not a kernel object but an ordinary process subjected to five independently-configurable kernel mechanisms — namespaces, cgroups, capabilities, a seccomp syscall filter, and a union-mounted root filesystem — and that removing any one still leaves something that runs, just less isolated. It grounds each mechanism in primary kernel documentation (the `namespaces(7)`, `cgroups(7)`, `capabilities(7)` and `seccomp(2)` man pages) and argues that isolation itself predates Docker: namespaces existed from 2002, cgroups from 2008, and LXC had combined the two into a usable manager by 2008. Docker's actual contribution, on this account, was separating that execution plane from a distribution plane it added — a layered, content-addressed image format plus a transport protocol — which turned deployment into a deterministic operation rather than a configuration checklist. The note then traces the Docker/Podman architectural split to one design choice: Docker centers on a root-owned background daemon reached over a REST API, while Podman is daemonless, execs the OCI runtime directly on each invocation, and was designed rootless from the start. It documents that rootless storage now runs on the kernel's native `overlay` driver on sufficiently recent kernels rather than the slower FUSE fallback that was mandatory when the foundational 2020 rootless walkthrough it cites was written, and that the main remaining rootless performance cost has shifted to userspace network emulation rather than storage. It also catalogues concrete evidence that the shared-kernel isolation boundary is weak in practice — a sequence of `runc` CVEs from 2019 through 2025 and a kernel-level Dirty Pipe escape — backed by an academic study finding user namespaces prevented 7 of 9 tested runtime-escape exploits.

The note does not claim containers match VM-grade isolation; it states the isolation boundary is the entire kernel syscall surface and treats privileged containers as impossible to secure. It marks several figures as inference or explicitly unverified rather

than established fact: vendor-reported vulnerability-scanning percentages in the 64–90% range are flagged as coming from vendors with a commercial interest and reported without independent corroboration, and a widely repeated claim about differing rootless-capability counts between Podman and Docker was deliberately excluded because it traced to secondary blogs with no primary citation. It leaves open which network backend is default in the specific Podman release a reader will install (the note's own "Grounding gap" section tracks this as unsettled, though a 2026-09-29 addendum resolves it in favor of `pasta`), and it notes that its per-namespace kernel-introduction versions now rest on secondary sources because the current man page dropped its version column. The note presents itself as a living document carrying dated corrections rather than a final account, and it does not judge whether Docker's or Podman's architecture is better in absolute terms — only that each buys a different set of guarantees at a different cost.

Referenced sources

- [namespaces\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#), Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#), Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#), Docker Inc. **HIGH**
- [Docker overview](#), Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#), Docker Inc. **HIGH**
- [Docker build cache](#), Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#), The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#), The crun project / containers organisation **HIGH**

- [What happens behind the scenes of a rootless Podman container?](#), Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#), Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#), Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#), Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#), Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#), Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#), Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#), Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#), Poul-Henning Kamp **HIGH**
- [Namespaces](#), Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#), Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#), Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#), opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#), Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#), Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#), Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#), Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#), Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#), GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025 \(DOI, stable alternative locator\)](#), GigaScience (2021) **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#), arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#), WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#), Syl Taylor (2023) **MEDIUM**

- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#), Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#), Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#), The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#), Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#), Microsoft Learn **HIGH**
- [Install Docker Engine](#), Docker Inc. **HIGH**
- [Podman Installation Instructions](#), The Podman project **HIGH**
- [podman-machine\(1\)](#), The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#), Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#), The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11 — overlayfs updates for Linux 5.11](#), Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#), Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#), The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [podman/docs/tutorials/performance.md](#), The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [WSL2-Linux-Kernel releases](#), Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#), Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCf org move, what should the final name be? — Podman discussion #28699](#), Podman core maintainers (2026) **HIGH**

2.27 V28 — the Linux kernel mechanisms and history underlying Docker and Podman

The note's central claim is that a container is not a kernel object at all but an ordinary process that the kernel has been asked, through five independent and separately configurable interfaces, to lie to about what it can see and consume: namespaces, cgroups, capabilities, a seccomp syscall filter, and a union-mounted root filesystem. Each mechanism is documented from the Linux manual pages and the OCI image and runtime specifications rather than from vendor marketing, and the note stresses that removing

any single one still leaves something that runs, just less isolated. It backs a historical argument with primary sources: isolation via `chroot`, BSD jails, Solaris Zones and LXC predates Docker by years, and the mount namespace (2002) and cgroups (2008) were already in the kernel before Docker's 2013 open-source release, so Docker's real contribution was pairing existing isolation primitives with a layered, content-addressed image format and a registry transport protocol — a distribution solution, not an isolation one. On top of that history, it documents Docker's daemon-centred client-server architecture against Podman's daemonless fork-exec model built on `crun` and `common`, and supports its security guidance — prefer rootless, never use `--privileged` — with a named sequence of `runc` CVEs and an academic study finding user namespaces prevented most tested container-escape exploits.

The note is careful about what it does not assert as settled. It excludes, by name, a widely repeated but unsourced comparison of exact kernel-capability counts between Docker and Podman, having traced every instance back to secondary blogs without a primary citation, and names the experiment — reading the two projects' own capability-set sources or running `capsh --print` — that would be needed before trusting the figure. It flags `crun`'s own performance benchmark, a macOS bind-mount slowdown measurement, and a rootless-networking throughput figure as self-reported, single-study, or not independently replicated rather than as established fact. It records, rather than silently resolves, a case where two of Podman's own documentation pages state different kernel-version thresholds for native overlay support. And it treats one earlier open question — which network backend Podman defaults to for rootless containers — as closed only as of a dated 2026-09-29 correction, confirming the current default is `pasta` rather than the `slirp4netns` used in the maintainer walkthrough the note otherwise quotes.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**

- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\)](#) — Podman manual page — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman](#) — project README — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun](#) — project README — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**

- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [DOI 10.1093/gigascience/giab025](#) — stable alternative locator for the GigaScience article above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**

- [CNCF org move, what should the final name be? — Podman discussion #28699](#) — Podman core maintainers (2026) **HIGH**

2.28 V29 — Containers with Docker and Podman

This note's central claim is that a container is not a kernel object but an ordinary process that the kernel has been asked, through five independent and separately-configured mechanisms, to lie to about what it can see and consume: namespaces (what it sees), cgroups (what it can consume), capabilities (how much of root it keeps), seccomp (which syscalls it may issue), and a union filesystem (what it sees as `/`). Removing any one still leaves something that runs, just with less isolation. On that foundation the note traces Docker and Podman as two different answers to the same kernel primitives: Docker's contribution was distribution — a layered, content-addressed image format and a registry protocol, wrapped around a client-server daemon (`dockerd` → `containerd` → `runc`) — while Podman keeps the same OCI image format and a Docker-compatible CLI but discards the daemon in favor of a fork-exec model, aimed at rootless-by-default operation. Evidence throughout is drawn from primary sources: the Linux kernel man pages for namespaces, cgroups, capabilities and seccomp; the OCI image and runtime specifications; vendor documentation from Docker and Red Hat/Podman; first-person accounts from engineers who built Docker and Podman; and, for the security claims, CVE advisories and one peer-reviewed study of 59 container-runtime CVEs finding that enabling user namespaces prevented most tested escape exploits.

The note is explicit about what rests on weaker footing. A `crun`-vs-`runc` performance comparison is flagged as the `crun` project's own self-reported, self-selected benchmark. Vendor claims about the prevalence of vulnerable images in the 64–90% range are marked unverified, since they come from security vendors with a commercial interest in the figure. A widely-repeated claim that Podman's rootless containers get a specific fixed number of capabilities fewer than Docker's was deliberately excluded as ungrounded — every instance traced back to secondary blogs with no primary citation. The note also records an unresolved internal disagreement between Podman's own documentation pages over which kernel version (5.12 vs. 5.13) is the threshold for native overlay support, and it does not independently verify exact per-namespace kernel introduction versions, noting that the current man page no longer carries that information. Several inferential steps — such as why Dockerfile instruction ordering matters, or why a rootless daemonless model suits a teaching context — are explicitly labeled as the author's inference rather than sourced claims.

Referenced sources

- [namespaces\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project — **HIGH**
- [cgroups\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project — **HIGH**
- [capabilities\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project — **HIGH**
- [seccomp\(2\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project — **HIGH**
- [OCI Image Format Specification](#), Open Container Initiative — **HIGH**
- [OCI Runtime Specification](#), Open Container Initiative — **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#), Open Container Initiative — **HIGH**
- [Seccomp security profiles for Docker](#), Docker Inc. — **HIGH**
- [Use the OverlayFS storage driver](#), Docker Inc. — **HIGH**
- [Docker overview](#), Docker Inc. — **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#), Docker Inc. — **HIGH**
- [Docker build cache](#), Docker Inc. — **HIGH**
- [podman\(1\) — Podman manual page](#), The Podman project / Podman Container Tools (CNCF) — **HIGH**
- [podman-container-tools/podman — project README](#), The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [containers/crun — project README](#), The crun project / containers organisation — **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#), Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) — **HIGH**
- [What is Podman?](#), Red Hat (2026) — **MEDIUM**
- [Why strace doesn't work in Docker](#), Julia Evans (2020) — **HIGH**
- [From dotCloud to Docker](#), Jérôme Petazzoni (2017) — **HIGH**
- [Docker grows to harbor innovation](#), Alex Handy / SD Times (2014) — **MEDIUM**
- [A Pivot that Worked: The Docker Story](#), Dan Scholnick / Trinity Ventures (2019) — **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#), Stefano Noferi (2013) — **MEDIUM**
- [History of Containers](#), Antranig Vartanian — **MEDIUM**
- [Jails - High value but shitty Virtualization](#), Poul-Henning Kamp — **HIGH**

- [Namespaces](#), Iustin Pop (2019) — **MEDIUM**
- [A container-confinement breakout](#), Jake Edge / LWN.net (2019) — **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#), Amazon Web Services (2019) — **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#), opencontainers/runc maintainers — **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#), Sysdig (2025) — **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#), Cloud Native Computing Foundation (2025) — **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#), Red Hat Product Security — **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#), Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) — **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#), Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) — **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#), GigaScience (2021) — **HIGH**
- [DOI: 10.1093/gigascience/giab025](#), GigaScience (2021), stable alternative locator — **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#), arXiv preprint — **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#), WarpBuild — **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#), Syl Taylor (2023) — **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#), Docker for Mac issue tracker — **MEDIUM**
- [Docker on macOS is slow and how to fix it](#), Paolo Mainardi / CNCF (2023) — **MEDIUM**
- [Lima — Linux virtual machines](#), The Lima project (CNCf sandbox) — **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#), Abiola Ibrahim / the Colima project — **HIGH**
- [Get started with Docker remote containers on WSL 2](#), Microsoft Learn — **HIGH**
- [Install Docker Engine](#), Docker Inc. — **HIGH**
- [Podman Installation Instructions](#), The Podman project — **HIGH**
- [podman-machine\(1\)](#), The Podman project — **HIGH**

- [Ivan Velichko — articles on containers, Kubernetes and server-side](#), Ivan Velichko — **MEDIUM**
- [multiarch/qemu-user-static](#), The multiarch project — **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11, Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) — **HIGH**
- [Podman is gaining rootless overlay support](#), Daniel Walsh / Red Hat — Enable Sysadmin (2021) — **HIGH**
- [podman/rootless.md](#), The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [podman/docs/tutorials/performance.md](#), The Podman project / podman-container-tools organisation (CNCF) — **HIGH**
- [WSL2-Linux-Kernel releases](#), Microsoft (2024–2026) — **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#), Naoki Matsumoto and Akihiro Suda (2024) — **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699, Podman core maintainers (2026) — **HIGH**

2.29 V30 — Container Isolation and the Docker/Podman Architecture Split

This note establishes that a container is not a kernel object but an ordinary process placed under five independently developed kernel mechanisms — namespaces, cgroups, capabilities, seccomp filtering, and a union-mounted root filesystem — and that removing any one of the five still leaves something that runs, just with less isolation. It traces the lineage with primary sources rather than the usual clean-chain story: Kirk McKusick's SCCS-log dating of `chroot` to 1982, Poul-Henning Kamp's own account of FreeBSD jails (1999–2000) and their shared-hosting motive, Solaris Zones (2004–05), and Linux namespaces and cgroups, which the note argues are parallel, cross-pollinated responses rather than a single inherited design. Docker's own contribution is framed narrowly as distribution — a layered, content-addressed image format plus a registry protocol — since the isolation primitives it relies on already existed. The note then contrasts two architectures built on the same OCI image and runtime specifications: Docker's root-owned daemon, `containerd` and `runc` chain, and Podman's daemonless fork-exec model, which uses a shared pause process and the user namespace to run rootless by default. The security claims are backed concretely — CVE-2019-5736, CVE-2024-21626, the November 2025 `runc` advisory trio, and Dirty Pipe are each traced to a specific mechanism, and a peer-reviewed study of 59 CVEs across 11 runtimes anchors the claim that enabling user namespaces prevented 7 of 9 tested escape exploits.

The note is explicit about what it does not claim. It does not assert that containers match VM-level isolation — the boundary is the entire kernel syscall surface, weaker than a hypervisor's — and treats that as a cost to be evidenced, not asserted. Several points are flagged as unsettled rather than resolved: the causal chain from jails and Zones to Linux namespaces is marked as inference rather than established fact; Podman's own two documentation pages disagree by one kernel release (5.12 vs. 5.13) on the threshold for native rootless overlay support, and the note records that disagreement instead of picking a side; a widely repeated secondary-source claim about Podman defaulting to fewer capabilities than Docker is deliberately excluded for lack of any primary source; and performance figures such as `crun`'s benchmark against `runc`, ARM-emulation build slowdowns, and macOS bind-mount penalties are each marked as self-reported or single-study rather than independently corroborated. The note does not take a side in Docker versus Podman generally — it frames the difference as an architectural trade-off between a central daemon and a daemonless, least-privilege default, and confines itself to naming the evidence behind each specific cost.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**

- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [DOI: 10.1093/gigascience/giab025](#) — stable alternative locator for the GigaScience article above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**

- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.30 V31 — Containers as Linux Kernel Primitives Assembled by Docker and Podman

This note's central claim is that a container is not a distinct kernel object but an ordinary process subjected to five independent, separately-documented kernel mechanisms: namespaces (what the process can see), cgroups (what it can consume), capabilities (how much of root it keeps), seccomp (which syscalls it may issue), and a union-moun-

ted filesystem (what it sees as `/`). Each mechanism is backed by its own kernel manual page and its own history — namespaces and cgroups predate Docker by years, and LXC had already combined them into a working container manager before Docker existed. The note argues that Docker's real contribution was not isolation, which was already available, but distribution: a layered, content-addressed image format plus a transport protocol, built by a failing PaaS company that later open-sourced its container engine against its own board's wishes. Podman is presented as a second-generation correction to Docker's architecture rather than its kernel usage — removing the root-owned background daemon in favor of a fork-exec model, while keeping the same OCI image format, registry protocol and a Docker-compatible CLI. The note traces this lineage, the internals of each of the five mechanisms, and the OCI image and runtime specifications that make the two engines interchangeable, using kernel man pages, the OCI specifications themselves, vendor documentation, and first-person accounts from Docker's and Podman's own engineers as evidence.

The note is explicit about what it does not claim. It does not claim containers match the isolation of a hypervisor; it states plainly that sharing a kernel means the attack surface is the kernel's entire syscall interface, and documents several runc CVEs and a Dirty Pipe kernel-level escape as evidence that this is not theoretical. Several quantitative claims are flagged with their own uncertainty markers rather than presented as settled: macOS emulation and bind-mount slowdown figures are marked as reported rather than independently verified, and a widely-repeated comparison of Docker's versus Podman's default capability counts is deliberately excluded because every source for it traced back to uncited secondary blogs. The note also records an unresolved disagreement between Podman's own two documentation pages over which kernel version is required for native rootless overlay support (5.12 vs. 5.13), and leaves open which network backend is default in the exact release a reader will install, rather than resolving these questions itself.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**

- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**

- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [DOI 10.1093/gigascience/giab025](#) — stable alternative locator for the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**

- [CNCF org move, what should the final name be? — Podman discussion #28699](#) — Podman core maintainers (2026) HIGH

2.31 V32 — The Five-Mechanism Model of a Linux Container

The note's central claim is that a container is not a kernel object — there is no `struct container`, no dedicated syscall, no single subsystem that owns the concept — but an ordinary process that the kernel has been asked, through five independent and separately-documented interfaces, to lie to about what it can see and how much it can consume: namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it keeps), seccomp (which syscalls it may issue), and a union filesystem (what it sees as its root). Removing any one leaves something that still runs, just with less isolation. The note sources this model to primary kernel documentation (the `namespaces(7)`, `cgroups(7)`, `capabilities(7)` and `seccomp(2)` manual pages) and to the OCI image and runtime specifications, then traces the mechanisms' history as parallel, loosely connected developments rather than one inherited lineage — `chroot`, FreeBSD jails, Solaris Zones and Linux namespaces and cgroups arose independently from shared-hosting pressure between 1982 and 2008. Docker and Podman are framed as programs that issue these five kinds of kernel request in a repeatable order and package the filesystem the resulting process wakes up in; Docker's real contribution is credited to the distribution plane — an immutable, content-addressed image format plus a registry protocol — since the isolation primitives already existed. The two engines differ architecturally (a root-owned daemon reached over a REST API versus Podman's per-invocation fork-exec model with no daemon at all) while sharing the same OCI image format, registry protocol and interchangeable runtimes.

The note is explicit that this composition is bounded, not absolute: isolation rests on namespaces and cgroups rather than a hypervisor boundary, and the attack surface is the kernel's entire syscall interface — a claim it backs with named runtime-escape CVEs and kernel bugs that bypassed the container runtime entirely, plus one peer-reviewed study finding user namespaces prevented most tested escapes. It does not claim the five-mechanism model makes containers secure by default, nor that either engine is generally superior; the comparison is narrowed explicitly to who owns the container process. It marks several things as unsettled rather than resolved: the exact kernel version at which some namespace types first appeared, Podman's current default rootless network backend, and a frequently repeated capability-count comparison between the two engines that it declines to state for lack of a primary source.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**

- [Namespaces](#) — Justin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — GigaScience (2021), stable DOI alternative to the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**

- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko MEDIUM
- [multiarch/qemu-user-static](#) — The multiarch project MEDIUM
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) HIGH
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) HIGH
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) HIGH
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) HIGH
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) HIGH
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) MEDIUM
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) HIGH

2.32 V33 — Containers as a Composition of Independent Kernel Primitives

This note's central claim is that a container is not a kernel object at all: it is an ordinary process that namespaces, cgroups, capabilities, seccomp and a union filesystem have each been separately asked to constrain, and removing any one of the five still leaves something that runs. Each mechanism is traced to its own primary documentation — the kernel man pages for namespaces, cgroups, capabilities and seccomp, and the OCI image and runtime specifications for the image and `config.json` format — so the isolation half of the claim rests on citable, canonical sources rather than on Docker's own framing. The note further argues, and sources separately, that Docker's actual contribution was distribution rather than isolation: LXC already combined namespaces and cgroups by 2008, and what Docker added was a layered, content-addressed image format plus a registry protocol, a claim it backs with a dotCloud engineer's first-hand account of the pre-Docker `dc` prototype and with a contemporaneous 2013 technical write-up. It carries the engine architecture through in comparable detail — Docker's daemon-mediated client/containerd/runc chain versus Podman's daemonless fork-exec model with `common` and, by default, `crun` — using vendor documentation and a Red Hat maintainer's own walkthrough of a rootless Podman start.

The note is explicit about where its evidence runs out. Historical lineage claims are flagged as inference where sources disagree — for instance, that jails, Solaris Zones and Linux namespaces are parallel responses to shared-hosting pressure rather than one in-

herited design — and self-reported vendor benchmarks (`crun` versus `runc` timings) are marked as directional rather than independently measured. A "Grounding gap" section names claims it deliberately did not include: a widely repeated "11 versus 14 capabilities" comparison between Podman and Docker is excluded as untraceable to any primary source, and, as of the note's original writing, which rootless network backend Podman defaults to was left open pending a version check (a later correction records this settled at `pasta` as of Podman v6.1.2). Vulnerability-scanning figures above roughly 30% of images are marked unverified, attributed to vendors with a commercial interest in the number. The note does not claim containers match a hypervisor's isolation guarantee — it states the opposite, that the isolation boundary is the entire kernel system-call surface — and does not claim any engine's rootless mode is unconditionally safe, only that user-namespace use empirically prevented most tested escape exploits in the one systematic study it cites.

Referenced sources

- [namespaces\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#), Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification](#), Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#), Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#), Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#), Docker Inc. **HIGH**
- [Docker overview](#), Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#), Docker Inc. **HIGH**
- [Docker build cache](#), Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#), The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#), The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#), The crun project / containers organisation **HIGH**

- [What happens behind the scenes of a rootless Podman container?](#), Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#), Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#), Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#), Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#), Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#), Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#), Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#), Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#), Poul-Henning Kamp **HIGH**
- [Namespaces](#), Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#), Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#), Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#), opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#), Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#), Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#), Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#), Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#), Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#), GigaScience (2021) **HIGH**
- [DOI: 10.1093/gigascience/giab025 — stable alternative locator](#), GigaScience (2021) **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#), arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#), WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#), Syl Taylor (2023) **MEDIUM**

- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#), Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#), Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#), The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#), Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#), Microsoft Learn **HIGH**
- [Install Docker Engine](#), Docker Inc. **HIGH**
- [Podman Installation Instructions](#), The Podman project **HIGH**
- [podman-machine\(1\)](#), The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#), Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#), The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11 — overlayfs updates for Linux 5.11](#), Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#), Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#), The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [podman/docs/tutorials/performance.md](#), The Podman project / podman-container-tools organisation (CNCf) **HIGH**
- [WSL2-Linux-Kernel releases](#), Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#), Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCf org move, what should the final name be? — Podman discussion #28699](#), Podman core maintainers (2026) **HIGH**

2.33 V34 — Docker and Podman as Compositions of Independent Kernel Mechanisms

This note's central claim is that a container is not a kernel object at all but an ordinary process assembled from five independently-developed, independently-documented kernel mechanisms: namespaces (what the process can see), cgroups (what it can consume), capabilities (how much of root it retains), seccomp (which syscalls it may issue), and a union filesystem (what it sees as its root). Each mechanism is sourced to its own Linux manual page or the OCI image and runtime specifications, and the note traces

their history as three parallel, only loosely connected responses to shared-hosting pressure — `chroot`, FreeBSD jails, Solaris Zones, and Linux's own namespace and `cgroup` subsystems — rather than a single inherited lineage. It credits Docker's actual innovation as separating a distribution plane (a layered, content-addressed image format plus a registry transport) from an execution plane the kernel already provided, and traces Docker's own path from dotCloud's failing PaaS through its 2013 open-sourcing. Podman is presented as an architectural rebuttal rather than a rejection of Docker's kernel usage: same image format, same OCI runtime contract, same Docker-compatible CLI, but a per-invocation fork-exec model built around rootless operation from the start, in place of a root-owned background daemon.

The note is explicit about what it does not establish. It documents specific runtime-escape CVEs and a peer-reviewed study to argue that the isolation boundary is the entire Linux system-call surface — weaker than a hypervisor boundary — rather than claiming containers are a security equivalent to virtual machines. It flags a cross-runtime performance benchmark as self-reported by the project that produced it, and marks a widely repeated vulnerability-percentage figure for container images as unverified, since it traces to vendors with a commercial interest in the number. It leaves open the exact kernel version at which several individual namespace types were introduced, and — despite a correction pass — does not pin which userspace network backend is the current default for rootless Podman, keeping an older maintainer account as historical rather than folding it into the current-state narrative. It also explicitly rejects one frequently repeated comparison claim about capability-count differences between the two engines as unsupported by any primary source, rather than repeating it uncritically.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**

- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\)](#) — Podman manual page — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman](#) — project README — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun](#) — project README — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels](#) — runc (CVE-2024-21626) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**

- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — GigaScience (2021), stable DOI alternative to the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**

- [CNCF org move, what should the final name be? — Podman discussion #28699](#) — Podman core maintainers (2026) **HIGH**

2.34 V35 — The Container as a Composition of Independent Kernel Mechanisms

The note's central claim is that a container is not a kernel object at all: there is no `struct container` and no dedicated system call, only an ordinary process that five independent, separately-documented kernel interfaces have been configured to restrict — namespaces, cgroups, capabilities, seccomp, and a union filesystem for the root directory. Each mechanism is backed by its own primary source (the Linux manual pages for namespaces, cgroups, capabilities and seccomp, and the OCI image and runtime specifications for the packaging format), and the note argues that removing any one of the five still leaves something that runs, just with less isolation. On top of that mechanism layer, the note traces two historical claims: that Docker's real contribution was a distribution format (layered, content-addressed images plus a registry protocol) rather than isolation, which already existed via namespaces and cgroups years earlier; and that Podman's daemonless, fork-exec architecture is an argument against Docker's root-owned background daemon rather than against its use of the same underlying kernel primitives. Both claims are supported by primary-source and first-person testimony — kernel commit logs, the FreeBSD jails author's own account, a former dotCloud/Docker engineer's retrospective, and the Podman project's own documentation and manual pages.

The note is explicit about where its evidence runs out. It does not claim that containers match hypervisor-grade isolation; it states plainly that the shared-kernel attack surface is enormous and cites a peer-reviewed study, several CVEs, and a kernel-level exploit (Dirty Pipe) that bypassed the container runtime entirely as support for that limit. It also flags several unsettled points rather than resolving them: it declines to state Podman's exact default rootless network backend as of a particular release without checking, notes that two of Podman's own documentation pages give different kernel-version thresholds for native overlay support without picking one, and explicitly excludes a widely-repeated comparative claim about capability-count differences between Docker and Podman because every source for it traced back to secondary blogs with no primary citation. Passages marked `[inference]` or `[unverified]` in the note mark reasoning or figures the author does not treat as independently confirmed, including vendor-reported vulnerability-scan percentages presented only as a range from commercially interested parties. The note is descriptive throughout — a mechanism account rather than an argument for or against using containers.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**

- [Namespaces](#) — Justin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [DOI: 10.1093/gigascience/giab025](#) — stable alternative locator for the above — GigaScience (2021) **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**

- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.35 V36 — Docker and Podman as Configurations of Kernel Primitives

This note's central claim is that a container is not a distinct kernel object but an ordinary process that has been placed under five independent, separately-configurable mechanisms: namespaces (what it can see), cgroups (what it can consume), capabilities (how much of root it retains), seccomp (which syscalls it may issue), and a union filesystem (what it sees as its root). It backs each mechanism with the corresponding Linux manual page and the OCI image and runtime specifications, then uses that foundation to correct a common historical narrative: rather than a single lineage running `chroot` through jails and Zones into Linux namespaces, the note argues from primary accounts (including the FreeBSD jails author and a kernel-log-dated `chroot` history) that these were parallel responses to the same shared-hosting pressure, with cross-pollination rather than direct inheritance. On that basis it credits Docker's real contribution as distribution — a layered, content-addressed image format plus a registry protocol — layered over kernel isolation primitives that already existed, tracing that contribution to dotCloud's failing PaaS and its 2013 open-sourcing. Podman is presented not as a rejection of Docker's kernel usage but of its daemon architecture: same OCI image format and a Docker-compatible CLI, but a per-invocation fork-exec model instead of a root-owned daemon, with rootless operation via user namespaces built in from the start.

The note is careful about what it does not claim. It states directly that the isolation boundary containers provide is only as strong as the entire Linux system-call surface — weaker than a hypervisor — and supports this with named runtime-escape CVEs and a kernel bug that bypassed the container runtime layer entirely, plus a peer-reviewed study of tested escape exploits. Several figures are explicitly flagged as weaker evidence: a runtime performance comparison is marked as self-reported by the project that produced it, and a widely circulated image-vulnerability percentage is marked unverified because it originates from security vendors with a commercial interest in the figure. The note also names open questions rather than resolving them silently — exact per-namespace kernel introduction versions are not pinned to a primary source, and it records that two of the project's own documentation pages disagree by one kernel release on a rootless-storage threshold. It explicitly declines to repeat a frequently seen capability-count comparison between the two engines, stating that every instance it found traced back to secondary sources with no primary citation.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**

- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — GigaScience (2021), stable DOI alternative to the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**

- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.36 V37 — Container Isolation and Distribution via Docker and Podman

This note's central claim is that a container is not a kernel object but an ordinary process assembled from five independent, separately documented kernel mechanisms: namespaces control what the process can see, cgroups control what it can consume, capabilities control how much of root it retains, seccomp controls which syscalls it may issue, and a union filesystem controls what it sees as its root. Each mechanism is

sourced to its own Linux manual page and to the OCI image and runtime specifications, and the note argues that Docker's real contribution was not these kernel primitives — namespaces and cgroups already existed — but the separation of a distribution plane (a layered, content-addressed image format plus a registry protocol) from the execution plane. It traces this lineage through dotCloud's failing PaaS, Docker's 2013 open-sourcing, and the later split of `containerd` and `libcontainer` out of the engine, and it treats the common `chroot`→`jails`→`Zones`→`LXC`→`Docker` narrative with more precision than usual, presenting jails, Zones and Linux namespaces as parallel responses to shared-hosting pressure rather than a single inherited design. Podman is presented as an architectural rebuttal rather than a rejection of Docker's kernel usage: it keeps the OCI image format and a Docker-compatible CLI, but replaces the root-owned background daemon with a per-invocation fork-exec model (monitored by a small helper process, `conmon`) built around rootless operation via user namespaces from the outset.

The note is explicit about what it does not claim. It does not claim containers match a hypervisor's isolation boundary — the isolation is bounded by the entire Linux system-call surface, which it documents with specific runtime-escape CVEs and a peer-reviewed study finding that user namespaces prevented most tested escapes, rather than asserting containers are inherently secure. It marks weaker evidence explicitly: a runtime-speed comparison between two OCI runtimes is flagged as self-reported by the project that ran it, and a widely repeated image-vulnerability percentage is flagged unverified because it originates with vendors that have a commercial interest in the figure. It also records open questions it declines to resolve on its own authority — the exact kernel version at which several individual namespace types were introduced, and a discrepancy between two of Podman's own documentation pages over which kernel release is needed for a storage feature. And it explicitly rejects one frequently repeated comparison claim, a specific capability-count difference between the two engines, as unsupported by any primary source its research could find, rather than repeating it uncritically because other articles do.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**

- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\)](#) — Podman manual page — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman](#) — project README — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun](#) — project README — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails – High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**
- [Namespaces](#) — Iustin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/ runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**

- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — GigaScience (2021), stable DOI alternative to the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCF sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**
- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**

- [WSL2-Linux-Kernel releases](#) — Microsoft (2024–2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**

2.37 V38 — Docker and Podman Container Mechanics

The note argues that a container is not a kernel object but an ordinary process subjected to five independently configurable kernel mechanisms — namespaces, cgroups, capabilities, seccomp filtering, and a union-mounted root filesystem — and that Docker's real innovation was not isolation, which already existed, but a content-addressed image format plus a distribution protocol. It traces the lineage from 1982 BSD `chroot` through FreeBSD jails, Solaris Zones, and Linux namespaces and cgroups, arguing these are parallel responses to shared-hosting pressure rather than a single inherited design, and it recounts Docker's origin inside the failing PaaS company dotCloud and its 2013 open-sourcing. It sets Docker's client-daemon-containerd-`runc` architecture against Podman's daemonless fork-exec model built on `libpod`, `crun` and `common`, and it treats rootless operation — enabled by the user namespace's unique ability to grant `CAP_SYS_ADMIN` to an unprivileged user inside a new namespace — as the single highest-leverage security improvement, citing a peer-reviewed study finding user namespaces prevented seven of nine tested escape exploits. Each mechanism is backed by kernel man pages or the OCI specifications; historical claims draw on primary sources including a BSD `chroot` author's own account, a Red Hat maintainer walkthrough, and Docker and Podman engineers' own retrospectives.

The note is explicit that the isolation boundary is the entire kernel syscall interface, "weaker than the separation of a hypervisor," and it catalogs several `runc` CVEs plus one kernel bug (Dirty Pipe) as concrete escapes no runtime fix could have prevented, concluding that privileged containers "are simply never going to be truly secure." Several points are flagged as unsettled or self-reported rather than independently verified: `crun`'s performance numbers against `runc` are the project's own benchmark; vendor claims of 64–90% vulnerable images are noted but not corroborated; and Podman's two own documentation pages disagree by one kernel release (5.12 vs. 5.13) on when native rootless overlay mounts became safe, a disagreement the note records rather than resolves. A "Grounding gap" section lists claims deliberately excluded for lacking primary sourcing — notably a widely repeated but uncorroborated comparison of Podman's versus Docker's default rootless capability counts — and the note states plainly that it does not settle exact per-namespace kernel introduction versions, since the current man page dropped its version column.

Referenced sources

- [namespaces\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [cgroups\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [capabilities\(7\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [seccomp\(2\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [OCI Image Format Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification](#) — Open Container Initiative **HIGH**
- [OCI Runtime Specification — Linux Container Configuration](#) — Open Container Initiative **HIGH**
- [Seccomp security profiles for Docker](#) — Docker Inc. **HIGH**
- [Use the OverlayFS storage driver](#) — Docker Inc. **HIGH**
- [Docker overview](#) — Docker Inc. **HIGH**
- [Run the Docker daemon as a non-root user \(Rootless mode\)](#) — Docker Inc. **HIGH**
- [Docker build cache](#) — Docker Inc. **HIGH**
- [podman\(1\) — Podman manual page](#) — The Podman project / Podman Container Tools (CNCF) **HIGH**
- [podman-container-tools/podman — project README](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [containers/crun — project README](#) — The crun project / containers organisation **HIGH**
- [What happens behind the scenes of a rootless Podman container?](#) — Daniel Walsh and Giuseppe Scrivano / Red Hat (2020) **HIGH**
- [What is Podman?](#) — Red Hat (2026) **MEDIUM**
- [Why strace doesn't work in Docker](#) — Julia Evans (2020) **HIGH**
- [From dotCloud to Docker](#) — Jérôme Petazzoni (2017) **HIGH**
- [Docker grows to harbor innovation](#) — Alex Handy / SD Times (2014) **MEDIUM**
- [A Pivot that Worked: The Docker Story](#) — Dan Scholnick / Trinity Ventures (2019) **MEDIUM**
- [What Docker adds to the Linux kernel primitives](#) — Stefano Noferi (2013) **MEDIUM**
- [History of Containers](#) — Antranig Vartanian **MEDIUM**
- [Jails - High value but shitty Virtualization](#) — Poul-Henning Kamp **HIGH**

- [Namespaces](#) — Justin Pop (2019) **MEDIUM**
- [A container-confinement breakout](#) — Jake Edge / LWN.net (2019) **HIGH**
- [Anatomy of CVE-2019-5736: A runc container escape!](#) — Amazon Web Services (2019) **HIGH**
- [runc security advisory GHSA-9493-h29p-rfm2 \(CVE-2025-31133\)](#) — opencontainers/runc maintainers **HIGH**
- [New runc vulnerabilities allow container escape: CVE-2025-31133, CVE-2025-52565, CVE-2025-52881](#) — Sysdig (2025) **MEDIUM**
- [runc container breakout vulnerabilities: A technical overview](#) — Cloud Native Computing Foundation (2025) **HIGH**
- [RHSB-2024-001 Leaky Vessels — runc \(CVE-2024-21626\)](#) — Red Hat Product Security **HIGH**
- [Towards Improving Container Security by Preventing Runtime Escapes](#) — Michael Reeves, Dave (Jing) Tian, Antonio Bianchi, Z. Berkay Celik (2021) **HIGH**
- [Escaping containers using the Dirty Pipe vulnerability](#) — Eric Mountain, Christophe Tafani-Dereeper, Tommy McCormick, Frederic Baguelin / Datadog Security Labs (2022) **HIGH**
- [An analysis of security vulnerabilities in container images for scientific data analysis](#) — GigaScience (2021) **HIGH**
- [10.1093/gigascience/giab025](#) — DOI stable alternative locator for the above **HIGH**
- [On the Relation Between Outdated Docker Containers, Severity Vulnerabilities, and Bugs](#) — arXiv preprint **MEDIUM**
- [Moving ARM64 Docker Builds Off QEMU](#) — WarpBuild **MEDIUM**
- [Embracing the Future with Multi-Arch Containers](#) — Syl Taylor (2023) **MEDIUM**
- [Rosetta emulation on arm64 macOS breaks many amd64 images \(docker/for-mac#7137\)](#) — Docker for Mac issue tracker **MEDIUM**
- [Docker on macOS is slow and how to fix it](#) — Paolo Mainardi / CNCF (2023) **MEDIUM**
- [Lima — Linux virtual machines](#) — The Lima project (CNCf sandbox) **HIGH**
- [Colima — container runtimes on macOS and Linux with minimal setup](#) — Abiola Ibrahim / the Colima project **HIGH**
- [Get started with Docker remote containers on WSL 2](#) — Microsoft Learn **HIGH**
- [Install Docker Engine](#) — Docker Inc. **HIGH**
- [Podman Installation Instructions](#) — The Podman project **HIGH**
- [podman-machine\(1\)](#) — The Podman project **HIGH**
- [Ivan Velichko — articles on containers, Kubernetes and server-side](#) — Ivan Velichko **MEDIUM**

- [multiarch/qemu-user-static](#) — The multiarch project **MEDIUM**
- [Merge tag ovl-update-5.11](#) — overlayfs updates for Linux 5.11 — Miklos Szeredi, merged by Linus Torvalds / Linux kernel (2020) **HIGH**
- [Podman is gaining rootless overlay support](#) — Daniel Walsh / Red Hat — Enable Sysadmin (2021) **HIGH**
- [podman/rootless.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [podman/docs/tutorials/performance.md](#) — The Podman project / podman-container-tools organisation (CNCF) **HIGH**
- [WSL2-Linux-Kernel releases](#) — Microsoft (2024-2026) **HIGH**
- [bypass4netns: Accelerating TCP/IP Communications in Rootless Containers](#) — Naoki Matsumoto and Akihiro Suda (2024) **MEDIUM**
- [CNCF org move, what should the final name be?](#) — Podman discussion #28699 — Podman core maintainers (2026) **HIGH**